CS23532- COMPUTER NETWORKS



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23532- COMPUTER NETWORKS LAB MANUAL

SECOND YEAR

THIRD SEMESTER

2026- 2027

ODD SEMESTER

NAME: **HARISH.R**

REG NO: **251901056**

Ex.No.1	**Study of various Network commands used in Linux and Windows: Hands-on practice of various network commands.**
Date:	

AIM: - To Study of various Network commands used in Linux and Windows

S.No	Command	Synopsis	Description	Sample Input	Sample Output
1	`arp -a`	`arp -a`	Displays the ARP cache, showing the IP addresses and corresponding MAC addresses of devices on the local network.	`arp -a`	`Interface: 192.168.1.10 --- 0x7 Internet Address Physical Address Type 192.168.1.1 00-1a-2b-3c-4d-5e dynamic`
2	`hostname`	`hostname`	Displays the hostname (computer name) of the local system.	`hostname`	`DESKTOP-PC01`
3	`ipconfig /all`	`ipconfig /all`	Displays complete TCP/IP configuration, including IP address, subnet mask, gateway, DNS, DHCP, and MAC address.	`ipconfig /all`	`IPv4 Address : 192.168.1.10 Default Gateway : 192.168.1.1 DNS Server : 8.8.8.8 DHCP Enabled : Yes`
4	`nbtstat -a`	`nbtstat -a <hostname>`	Displays NetBIOS name table of a remote computer for troubleshooting NetBIOS over TCP/IP.	`nbtstat -a DESKTOP-PC01`	`NetBIOS Remote Machine Name Table Name Type Status DESKTOP-PC01 <00> UNIQUE Registered`
5	`netstat`	`netstat`	Displays active TCP/IP connections, listening ports, routing tables, and interface statistics.	`netstat`	`Proto Local Address Foreign Address State TCP 192.168.1.10:49762 142.250.190.78:443 ESTABLISHED`

6	netstat - r	netstat -r	Displays the IP routing table of the local computer.	netstat -r	IPv4 Route Table Network Destination Netmask Gateway Interface
S.No	**Command**	**Synopsis**	**Description**	**Sample Input**	**Sample Output**
7	nslookup	nslookup <domain>	Queries the DNS server to obtain the IP address or DNS records of a domain.	nslookup www.google.com	Server: dns.google Address: 8.8.8.8 Name: www.google.com Address: 142.250.190.68
8	pathping	pathping <hostname/IP>	Combines the features of ping and tracert to trace the route and measure packet loss at each hop.	pathping www.google.com	Tracing route to www.google.com Computing statistics... Hop 1 192.168.1.1 Hop 2 10.0.0.1 Packet Loss: 0%
9	ping	ping <hostname/IP>	Tests connectivity between the local computer and another host using ICMP Echo Request and Reply messages.	ping www.facebook.com	Pinging www.facebook.com [157.240.22.35] with 32 bytes of data: Reply from 157.240.22.35: bytes=32 time=25ms TTL=54
10	route	route print	Displays or modifies the IP routing table. It is commonly used to view or configure static routes.	route print	IPv4 Route Table Active Routes: Network Destination Netmask Gateway Interface

Some important Linux networking commands

S.No	Command	Synopsis	Description	Sample Input	Sample Output

1	ip	ip <OPTIONS> <OBJECT> <COMMAND>	Displays and manages IP addresses, routing tables, and	ip address show	2: eth0: <UP,BROADCAST,RUNNING> inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
S.No	**Command**	**Synopsis**	**Description**	**Sample Input**	**Sample Output**
			network interfaces.		
2	ip address add	ip address add <IP>/<Prefix> dev <Interface>	Assigns an IP address to a network interface.	ip address add 192.168.1.254/24 dev eth0	Assigned 192.168.1.254/24 to eth0 successfully.
3	ip address del	ip address del <IP>/<Prefix> dev <Interface>	Removes an IP address from a network interface.	ip address del 192.168.1.254/24 dev eth0	Deleted 192.168.1.254/24 from eth0.
4	ip link set up	ip link set <Interface> up	Brings the network interface online.	ip link set eth0 up	Interface eth0 is UP.
5	ip link set down	ip link set <Interface> down	Brings the network interface offline.	ip link set eth0 down	Interface eth0 is DOWN.
6	ip link set promisc on	ip link set <Interface> promisc on	Enables promiscuous mode on the interface.	ip link set eth0 promisc on	Promiscuous mode enabled on eth0.
7	ip route add default	ip route add default via <Gateway> dev <Interface>	Adds the default gateway for all outgoing traffic.	ip route add default via 192.168.1.254 dev eth0	Default route added successfully.
8	ip route add	ip route add <Network> via <Gateway>	Adds a static route through a gateway.	ip route add 192.168.1.0/24 via 192.168.1.254	Route added successfully.
9	ip route add (Direct Route)	ip route add <Network> dev <Interface>	Adds a direct route through a network interface.	ip route add 192.168.1.0/24 dev eth0	Route added via eth0.
10	ip route delete	ip route delete <Network> via <Gateway>	Deletes a static route.	ip route delete 192.168.1.0/24 via 192.168.1.254	Route deleted successfully.

11	ip route get	`ip route get <Destination IP>`	Displays the route used to reach a destination.	`ip route get 10.10.1.4`	`10.10.1.4 via 192.168.1.254 dev eth0 src 192.168.1.100`
12	ifconfig	`ifconfig`	Displays	`ifconfig`	`eth0:`
S.No	**Command**	**Synopsis**	**Description**	**Sample Input**	**Sample Output**
			and configures network interface parameters (deprecated in modern Linux).		`flags=4163<UP,BROADCAST,RUNNING> inet 192.168.1.100 netmask 255.255.255.0`
13	mtr	`mtr <hostname/IP>`	Combines Ping and Traceroute to analyze network paths.	`mtr google.com`	`HOST: localhost Loss% Snt Last Avg Best Wrst StDev`
14	mtr -g	`mtr -g <hostname>`	Displays numeric IP addresses instead of hostnames.	`mtr -g google.com`	`192.168.1.1 → 142.250.190.78`
15	mtr -b	`mtr -b <hostname>`	Displays both hostnames and IP addresses.	`mtr -b google.com`	`router.local (192.168.1.1) google.com (142.250.190.78)`
16	mtr -c	`mtr -c <count> <hostname>`	Sends a specified number of probes.	`mtr -c 10 google.com`	`10 probes completed successfully.`
17	tcpdump -D	`tcpdump -D`	Lists all available network interfaces.	`tcpdump -D`	`1.eth0 2.lo 3.wlan0`
18	tcpdump	`tcpdump -i <Interface>`	Captures packets on a network interface.	`tcpdump -i eth0`	`Listening on eth0...`
19	tcpdump -c	`tcpdump -i <Interface> -c <count>`	Captures a specified number of packets.	`tcpdump -i eth0 -c 10`	`10 packets captured.`
20	tcpdump host	`tcpdump -i <Interface> host <IP>`	Captures traffic to and from a specific host.	`tcpdump -i eth0 host 8.8.8.8`	`Captured packets to/from 8.8.8.8.`
21	tcpdump	`tcpdump -i`	Captures	`tcpdump -i eth0`	`Source packets from 8.8.8.8`

S.No	Command	Synopsis	Description	Sample Input	Sample Output
	src host	<Interface> src host <IP>	packets originating from a specific host.	src host 8.8.8.8	displayed.
22	tcpdump dst host	tcpdump -i <Interface> dst host <IP>	Captures packets destined for a specific host.	tcpdump -i eth0 dst host 8.8.8.8	Destination packets to 8.8.8.8 displayed.
23	tcpdump net	tcpdump -i <Interface> net <Network>/<Prefix>	Captures traffic from a specified network.	tcpdump -i eth0 net 10.1.0.0/24	Packets from network 10.1.0.0/24 displayed.
24	tcpdump port	tcpdump -i <Interface> port <Port>	Captures traffic on a specific port.	tcpdump -i eth0 port 53	DNS packets captured.
25	tcpdump host and port	tcpdump -i <Interface> host <IP> and port <Port>	Captures traffic for a specific host and port.	tcpdump -i eth0 host 8.8.8.8 and port 53	DNS traffic with host 8.8.8.8 displayed.
26	tcpdump port not	tcpdump -i <Interface> port not <Port1> and not <Port2>	Captures all traffic except the specified ports.	tcpdump -i eth0 port not 53 and not 25	Traffic excluding ports 53 and 25 captured.
27	ping	ping <hostname/IP>	Tests network connectivity using ICMP Echo Requests and Echo Replies.	ping google.com	64 bytes from google.com: icmp_seq=1 ttl=117 time=10.5 ms
28	ping -c	ping -c <count> <hostname>	Sends a specified number of ICMP Echo Requests.	ping -c 10 google.com	10 packets transmitted, 10 received, 0% packet loss.

Configuring an Ethernet connection by using nmcli:

Step No.	Task	Command	Expected Result
1	List all Network Manager connection profiles	`nmcli connection show`	Displays all available network connection profiles with their **Name, UUID, Type, and Device**.
2	Create a new Ethernet connection profile It is specifically an **`nmcli` connection management command**. • **`nmcli`** → NetworkManager Command Line Interface • **`connection`** → Manages network connection profiles. • **`add`** → Creates a new connection profile.	`nmcli connection add con-name Internal-LAN ifname enp1s0 type ethernet`	Creates a new Ethernet connection profile named **Internal-LAN**.
3	Rename an existing connection profile (Optional)	`nmcli connection modify "Wired connection 1" connection.id Internal-LAN`	Renames the existing connection profile to **Internal-LAN**.
4	Display the current settings of the connection profile	`nmcli connection show "Internal-LAN"`	Displays interface name, autoconnect status, IPv4 and IPv6 configuration details.
5	Configure IPv4 using DHCP	`nmcli connection modify "Internal-LAN" ipv4.method auto`	Configures the interface to obtain the IPv4 address automatically using DHCP.
6	Configure a static IPv4 address	`nmcli connection modify "Internal-LAN" ipv4.method manual ipv4.addresses 192.0.2.1/24 ipv4.gateway 192.0.2.254 ipv4.dns 192.0.2.200 ipv4.dns-search example.com`	Assigns a static IPv4 address, gateway, DNS server, and DNS search domain.
7	Configure IPv6 using	`nmcli connection modify`	Enables automatic IPv6

Step No.	Task	Command	Expected Result
	SLAAC	`"Internal-LAN" ipv6.method auto`	configuration using Stateless Address Autoconfiguration (SLAAC).
8	Configure a static IPv6 address	`nmcli connection modify "Internal-LAN" ipv6.method manual ipv6.addresses 2001:db8:1::fffe/64 ipv6.gateway 2001:db8:1::ffee ipv6.dns 2001:db8:1::ffbb ipv6.dns-search example.com`	Assigns a static IPv6 address, gateway, DNS server, and search domain.
9	Activate the connection profile	`nmcli connection up Internal-LAN`	Activates the **Internal-LAN** network connection profile.
10	Verify the IP address configuration	`ip address show enp1s0`	Displays the assigned IPv4 and IPv6 addresses of the **enp1s0** network interface.
11	Verify the IPv4 default gateway	`ip route show default`	Displays the configured IPv4 default gateway.
12	Verify the IPv6 default gateway	`ip -6 route show default`	Displays the configured IPv6 default gateway.
13	Verify the DNS configuration	`cat /etc/resolv.conf`	Displays the configured DNS servers and DNS search domain.
14	Test network connectivity	`ping <host-name-or-IP-address>`	Sends ICMP Echo Requests to verify connectivity with the specified host.

Sample Verification Outputs

Verification Command	Sample Output
ip address show enp1s0	inet 192.0.2.1/24 inet6 2001:db8:1::fffe/64
ip route show default	default via 192.0.2.254 dev enp1s0 proto static
ip -6 route show default	default via 2001:db8:1::ffee dev enp1s0 proto static
cat /etc/resolv.conf	search example.com
Verification Command	**Sample Output**

	nameserver 192.0.2.200 nameserver 2001:db8:1::ffbb
ping google.com	64 bytes from google.com: icmp_seq=1 ttl=117 time=11.2 ms

OUTPUT:

```
student@fedora:~$ arp -a
? (10.0.2.0) at <incomplete> on enp0s3
_gateway (10.0.2.2) at 52:55:0a:00:02:02 [ether] on enp0s3
student@fedora:~$ hostname
fedora
student@fedora:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::55f0:9771:cb55:7bf3  prefixlen 64  scopeid 0x20<link>
        inet6 fd17:625c:f037:2:eebe:9b47:484f:31ac  prefixlen 64  scopeid 0x0<global>
        ether 08:00:27:2d:91:bf  txqueuelen 1000  (Ethernet)
        RX packets 319  bytes 71231 (69.5 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 396  bytes 43565 (42.5 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 53  bytes 6056 (5.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 53  bytes 6056 (5.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
student@fedora:~$ netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp        0      0 fedora:bootpc           _gateway:bootps         ESTABLISHED
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags       Type       State         I-Node   Path
unix  3      [ ]         STREAM     CONNECTED     22973
unix  3      [ ]         STREAM     CONNECTED     23658    /run/systemd/journal/stdout
unix  2      [ ]         DGRAM      CONNECTED     17785
unix  2      [ ]         DGRAM      CONNECTED     20160    /run/chrony/chronyd.sock
unix  3      [ ]         STREAM     CONNECTED     20141
unix  3      [ ]         STREAM     CONNECTED     24041    /run/user/1000/gvfsd/socket-CpITnyiF
unix  3      [ ]         STREAM     CONNECTED     18329    /run/systemd/journal/stdout
unix  2      [ ]         DGRAM                    18039    /run/systemd/home/notify
unix  3      [ ]         STREAM     CONNECTED     23691
unix  3      [ ]         STREAM     CONNECTED     22632    /run/dbus/system_bus_socket
unix  3      [ ]         STREAM     CONNECTED     21242    /run/user/1000/bus
unix  3      [ ]         STREAM     CONNECTED     17756    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     22845    /run/user/1000/bus
unix  3      [ ]         STREAM     CONNECTED     23643
unix  3      [ ]         STREAM     CONNECTED     23073
unix  3      [ ]         STREAM     CONNECTED     18321    /run/dbus/system_bus_socket
unix  3      [ ]         STREAM     CONNECTED     23735    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     22514    /run/dbus/system_bus_socket
unix  3      [ ]         STREAM     CONNECTED     21418    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     28377
unix  3      [ ]         STREAM     CONNECTED     17890    /run/dbus/system_bus_socket
unix  3      [ ]         STREAM     CONNECTED     24102    @/tmp/.X11-unix/X0
unix  3      [ ]         STREAM     CONNECTED     23794
unix  3      [ ]         STREAM     CONNECTED     21401
unix  3      [ ]         STREAM     CONNECTED     21392
unix  3      [ ]         STREAM     CONNECTED     24911
unix  3      [ ]         STREAM     CONNECTED     23856
unix  3      [ ]         STREAM     CONNECTED     22821    @/tmp/.X11-unix/X0
unix  3      [ ]         STREAM     CONNECTED     22639
unix  2      [ ]         DGRAM      CONNECTED     19556
unix  3      [ ]         STREAM     CONNECTED     23040
unix  3      [ ]         STREAM     CONNECTED     21563    /run/systemd/journal/stdout
```

```
student@fedora:~$ netstat -r
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
default         _gateway        0.0.0.0         UG        0 0          0 enp0s3
10.0.2.0        0.0.0.0         255.255.255.0   U         0 0          0 enp0s3
student@fedora:~$ nslookup
> www.google.com
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   www.google.com
Address: 142.251.156.119
Name:   www.google.com
Address: 142.251.153.119
Name:   www.google.com
Address: 142.251.155.119
Name:   www.google.com
Address: 142.251.151.119
Name:   www.google.com
Address: 142.251.154.119
Name:   www.google.com
Address: 142.251.157.119
Name:   www.google.com
Address: 142.251.152.119
Name:   www.google.com
Address: 142.251.150.119
Name:   www.google.com
Address: 2001:4860:482d:7700::
Name:   www.google.com
Address: 2001:4860:4826:7700::
Name:   www.google.com
Address: 2001:4860:4829:7700::
Name:   www.google.com
Address: 2001:4860:482a:7700::
Name:   www.google.com
Address: 2001:4860:4827:7700::
Name:   www.google.com
Address: 2001:4860:482c:7700::
Name:   www.google.com
Address: 2001:4860:482b:7700::
Name:   www.google.com
Address: 2001:4860:4828:7700::
>
```

```
student@fedora:~$ ping google.com
PING google.com (142.250.77.110) 56(84) bytes of data.
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=1 ttl=255 time=10.6 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=2 ttl=255 time=7.34 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=3 ttl=255 time=7.23 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=4 ttl=255 time=20.1 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=5 ttl=255 time=7.47 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=6 ttl=255 time=8.16 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=7 ttl=255 time=14.6 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=8 ttl=255 time=7.13 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=9 ttl=255 time=7.11 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=10 ttl=255 time=6.63 ms
64 bytes from pnmaaa-aq-in-f14.1e100.net (142.250.77.110): icmp_seq=11 ttl=255 time=11.3 ms
^C
--- google.com ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10011ms
rtt min/avg/max/mdev = 6.630/9.794/20.100/4.015 ms
student@fedora:~$ route print
Usage: route [-nNvee] [-FC] [<AF>]          List kernel routing tables
       route [-v] [-FC] {add|del|flush} ...  Modify routing table for AF.

       route {-h|--help} [<AF>]              Detailed usage syntax for specified AF.
       route {-V|--version}                  Display version/author and exit.

        -v, --verbose            be verbose
        -n, --numeric            don't resolve names
        -e, --extend             display other/more information
        -F, --fib                display Forwarding Information Base (default)
        -C, --cache              display routing cache instead of FIB

  <AF>=Use -4, -6, '-A <af>' or '--<af>'; default: inet
  List of possible address families (which support routing):
    inet (DARPA Internet) inet6 (IPv6) ax25 (AMPR AX.25)
    netrom (AMPR NET/ROM) ipx (Novell IPX) ddp (Appletalk DDP)
    x25 (CCITT X.25)
student@fedora:~$
```

RESULT:

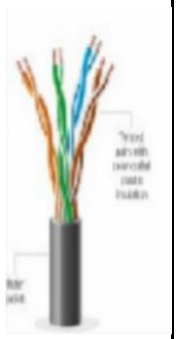
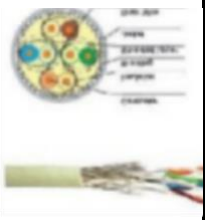
Thus, various networking commands in Linux and Windows were studied and their functions were verified successfully.

Ex.No.2	**Study of Network cables.**
Date:	1. Understand different types of Network cables. **2.** Make a cross-wired cable and straight through cable using clamping/crimping tool.

a) **Understand different types of network cable.**

Different type of cables used in networking are:

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) Cable
3. Coaxial Cable
4. Fibre Optic Cable

Cable type	**Category**	**Maximum Data Transmission**	**Advantages/ Disadvantages**	**Application/Use**	**Image**
UTP	Category 3	10 bps	**Advantages** • Cheaper in cost • Easy to install as they have a smaller overall diameter. **Disadvantages** • More prone to (EMI) Electromagnetic interference and noise	10Base-T Ethernet	
	Category 5	Up to 100 Mbps		Fast Ethernet, Gigabit Ethernet	
	Category 5e	1Gbps		Fast Ethernet, Gigabit Ethernet	
STP	Category6,6a	10Gbps	**Advantages** • Shielded. • Faster than UTP. • Less susceptible to noise and interference **Disadvantages** • Expensive • Greater installation effort	Gigabit Ethernet, 10G Ethernet (55m) Widely used in data centres	
SSTP	Category 7	10Gbps		Gigabit Ethernet, 10G Ethernet (100m)	

Coaxial cable	RG-6 RG-59 RG-11	10-100Mbps	• High bandwidth • Immune to interference • Low loss bandwidth • Versatile • **Disadvantages** • Limited distance • Cost • Size is bulky	Speed of signal is 500m Television network High speed internet connections	
fibre optics cable	Single mode Multi mode	100Gbps	**Advantages** • High speed • High bandwidth • High security • Long distance **Disadvantages** • Expensive • Requires skilled installers	• Maximum distance of fibre optics cable is around 100meters	

(b) Make Your Own Ethernet Cross-Over Cable/ Straight cable

Tools and parts needed:

- Ethernet cabling. CAT5e is certified for gigabit support, but CAT5 cabling works as well, just over shorter distances.
- A crimping tool. This is an all-in-one networking tool shaped to push down the pins in the plug and strip and cut the shielding off the cables.
- Two RJ45 plugs.
- Optional two plug shields.

Ethernet Cable (CAT5e/CAT5)

- CAT5e is recommended because it supports Gigabit Ethernet (1000 Mbps).
- CAT5 also works but supports lower speeds and shorter distances.

Crimping Tool

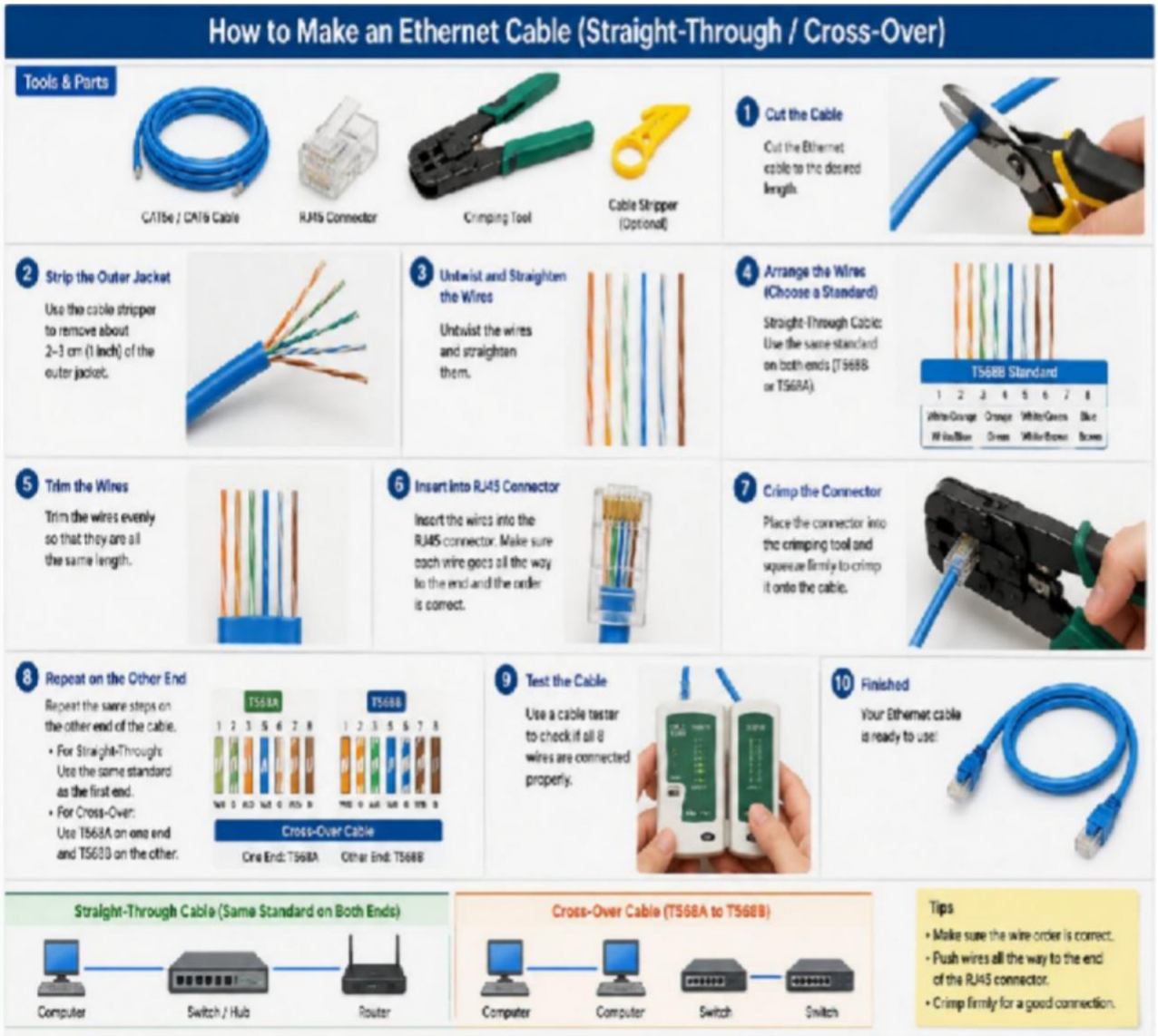
- A networking tool used to:
  - Strip the outer insulation of the cable.
  - Arrange and trim the wires.
  - Crimp (press) the RJ45 connector onto the cable.

RJ45 Connectors (2)

- Plastic connectors fitted at both ends of the Ethernet cable.
- Used to connect the cable to computers, switches, routers, etc.

🎬 Plug Shields (Optional)

- Protective covers placed over the RJ45 connectors.
- Prevent damage to the locking clip and reduce strain on the cable.



RESULT:

Thus, different types of network cables were studied, and straight-through and crossover Ethernet cables were prepared successfully.

Ex.No.3	**Experiments on CISCO PACKET TRACER (Simulation Tool):**
Date:	**a)** To understand environment of CISCO PACKET TRACER to design simple network. **b)** Analyse the behaviour of network devices using CISCO PACKET TRACER simulator. Design a simple network with multiple nodes and connect via networking devices available in library. Perform simulation and trace communication behaviour of specified network devices. 1: Use only HUB to design a small network having 4 to 6 hosts 2: Use only a Switch to design a small network with 4 to 6 hosts. 3: Use both the device (HUB and SWITCH) for a network and find out functioning difference between switch and hub. **Find out the network topology implemented in your college and draw and label that topology in your observation book.**

AIM: To study the Packet tracer tool Installation and analyse the behaviour of network devices using **CISCO PACKET TRACER**

INSTALLING PACKET TRACER

Windows

Visit the Cisco Networking Academy website (www.netacad.com).
Log in using your Cisco NetAcad account.
Download the Packet Tracer installer for Windows.
Run the setup file, accept the license agreement, and complete the installation.

Linux

Download the appropriate Packet Tracer package for your Linux distribution.

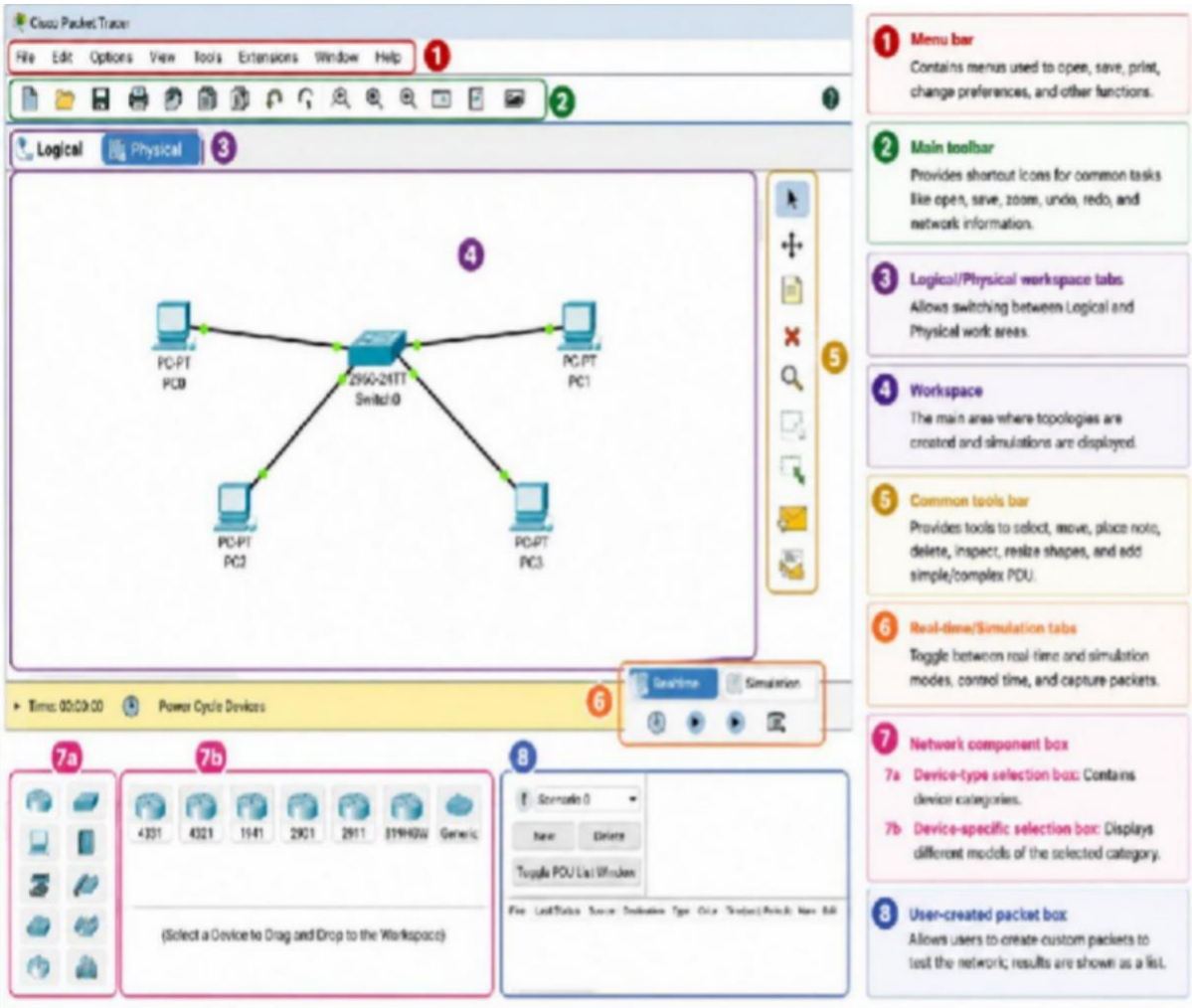
Open the terminal and make the installer executable:

chmod +x PacketTracer601_i386_installer-rpm.bin

Run the installer:

./PacketTracer601_i386_installer-rpm.bin

Follow the on-screen instructions to complete the installation.



b) Analyse the behaviour of network devices using CISCO PACKET TRACER simulator.

Scenario 1: Network Using Only a Hub

Procedure

Step 1

Open **Cisco Packet Tracer**.

Step 2

From **Network Devices**, drag a **Hub** to the workspace.

Step 3

Drag **4 PCs** (PC0, PC1, PC2, and PC3) from **End Devices**.

Step 4

Connect each PC to the Hub using **Copper Straight-Through** cables.

Step 5

Assign the following IP addresses.

Device	IP Address	Subnet Mask
PC0	192.168.1.1	255.255.255.0
PC1	192.168.1.2	255.255.255.0
PC2	192.168.1.3	255.255.255.0
PC3	192.168.1.4	255.255.255.0

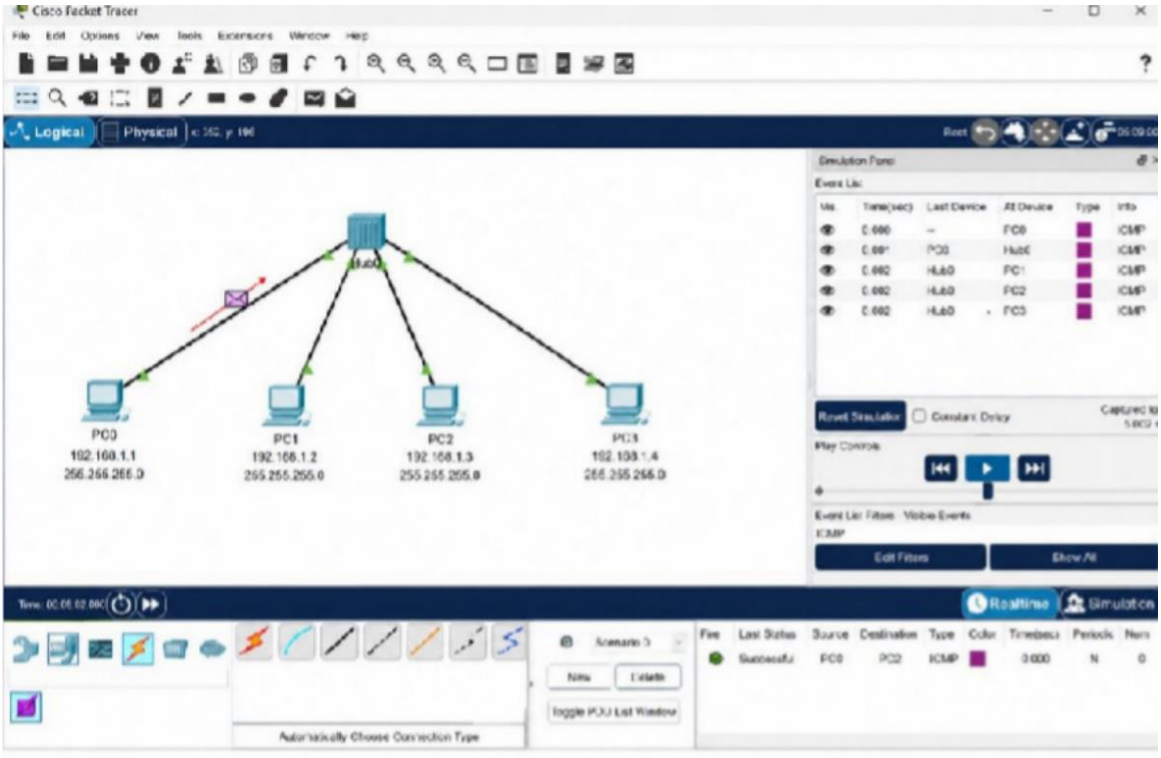
Step 6
Switch to **Simulation Mode**.

Step 7
Click **Add Simple PDU** and send a packet from **PC0** to **PC2**.

Step 8
Observe the packet movement.

Observation

- The Hub broadcasts the packet to **all connected hosts**.
- Every PC receives the packet.
- Only the destination PC accepts the packet.
- Network collisions may occur because all devices share the same communication channel.

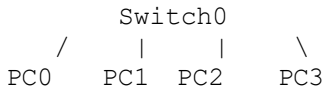


c) Scenario 2: Network Using Only a Switch

Aim

To design a simple network using a switch and observe the communication behaviour during simulation.

Network Topology



Devices Required

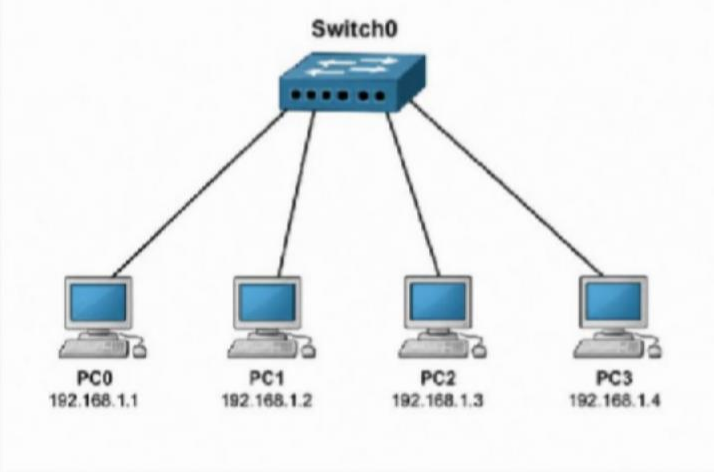
Device	Quantity
Switch 2960	1
PC-PT	4
Copper Straight-Through Cable	4

Procedure

Step 1
Open **Cisco Packet Tracer**.
Step 2
From **Network Devices → Switches**, drag a **2960 Switch** into the workspace.
Step 3
Drag four PCs (PC0, PC1, PC2, and PC3) from **End Devices**.
Step 4
Connect each PC to the switch using **Copper Straight-Through** cables.
Step 5
Assign the following IP addresses.

Device	IP Address	Subnet Mask
PC0	192.168.1.1	255.255.255.0
PC1	192.168.1.2	255.255.255.0
PC2	192.168.1.3	255.255.255.0
PC3	192.168.1.4	255.255.255.0

Step 6
Switch to **Simulation Mode**.
Step 7
Select **Add Simple PDU**.
Step 8
Click **PC0** and then **PC2**.
Step 9
Observe the packet movement.



Observation

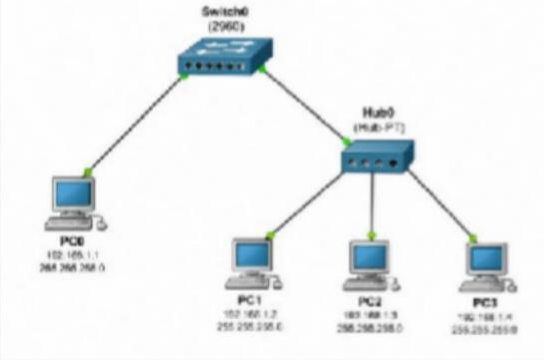
- The switch learns the MAC addresses of connected devices.
- Initially, the switch may flood the frame if the destination MAC address is unknown.
- Once the MAC address table is learned, the switch forwards packets **only to the destination port**.
- Other PCs do not receive the packet.
- No collisions occur because each switch port forms a separate collision domain.

Scenario 3: Network Using Both Hub and Switch

Aim

To design a network using both a hub and a switch and compare their communication behaviour.

Network Topology



Devices Required

Device	Quantity
Switch 2960	1
Hub	1
PC-PT	4

Device	Quantity
Copper Straight-Through Cable	5

Procedure

Step 1

Open Cisco Packet Tracer.

Step 2

Place one **2960 Switch** and one **Hub** in the workspace.

Step 3

Add four PCs.

Step 4

Connect:

PC0 → Switch

Hub → Switch

PC1 → Hub

PC2 → Hub

PC3 → Hub

Step 5

Assign the following IP addresses.

Device	IP Address	Subnet Mask
PC0	192.168.1.1	255.255.255.0
PC1	192.168.1.2	255.255.255.0
PC2	192.168.1.3	255.255.255.0
PC3	192.168.1.4	255.255.255.0

Step 6

Switch to **Simulation Mode**.

Step 7

Send a Simple PDU from **PC0** to **PC2**.

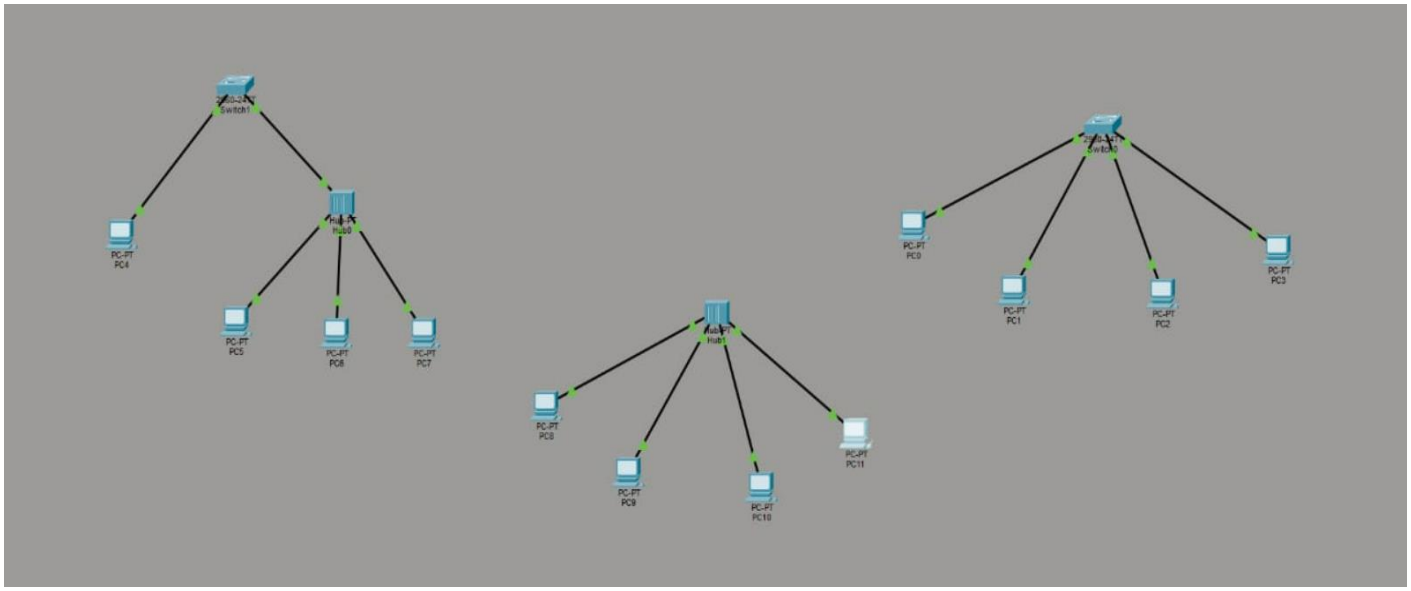
Step 8

Observe the packet movement.

Observation

- The switch forwards the frame only toward the hub.
- The hub broadcasts the frame to all devices connected to it.
- PC1 and PC3 receive the frame but discard it.
- PC2 accepts the packet because it is the intended destination.
- The switch limits unnecessary traffic, while the hub broadcasts within it

OUTPUT:



RESULT:

Thus, networks using a hub, switch, and both devices were designed and simulated successfully. Their communication behavior was observed and compared.

Ex.No.4	**a) Setup and configure a LAN (Local area network) using a Switch and Ethernet cables in your lab.** 1. Connect 3-4 host machines to a switch. 2. Assign ip addresses to each host machine. 3. Check the connectivity between the machines by using ping command. 4. Share and access files and folder across the machines of the LAN.
Date:	

LAN:

A local area network (LAN) is a collection of devices connected together in one physical location, such as a building, office, or home. A LAN can be small or large, ranging from a home network with one user to an enterprise network with thousands of users and devices in an office or school.

Regardless of size, a LAN's single defining characteristic is that it connects devices that are in a single, limited area.

Set Up a LAN (Local Area Network) Directly

How to Set Up a LAN (Local Area Network)

Step 1: Plan the Network

Plan and design an appropriate network topology based on the network requirements and the physical location of the devices.

Step 2: Gather the Required Devices

Prepare the following hardware components:

- 4 Computers (PCs)
- 1 Network Switch (8-port, 16-port, or 24-port)
- 4 Ethernet (UTP) Cables

Step 3: Connect the Devices

Connect each computer to the network switch using Ethernet cables. Plug one end of the Ethernet cable into the Ethernet port of the computer and the other end into an available port on the switch. Ensure that the link LEDs on the switch and the computers are ON, indicating successful connections.

Step 4: Assign IP Addresses to the PCs

1. Log in to each computer using an **Administrator** or **Owner** account.
2. Open **Control Panel → Network and Internet → Network and Sharing Center**.
3. Click **Change Adapter Settings**.
4. Right-click **Local Area Connection** (or **Ethernet**) and select **Properties**.
5. Select **Internet Protocol Version 4 (TCP/IPv4)** and click **Properties**.

6. Select the **Use the following IP address** option.
7. Enter the IP address and subnet mask for each computer as shown below:

Computer	IP Address	Subnet Mask
PC1	10.1.1.1	255.0.0.0
PC2	10.1.1.2	255.0.0.0
PC3	10.1.1.3	255.0.0.0
PC4	10.1.1.4	255.0.0.0

Click **OK** to save the configuration. Repeat the same procedure for all the PCs connected to the switch.

Step 5: Configure the Network Switch

1. Connect a computer to the switch using an Ethernet cable.
2. Open a web browser and enter the IP address of the switch in the address bar to access the switch's management interface.
3. Log in using the appropriate **username** and **password**.
4. Configure the basic settings of the switch.
5. Assign the following management IP address to the switch:

Device	IP Address	Subnet Mask
Switch	10.1.1.5	255.0.0.0

Save the configuration.

Step 6: Verify Network Connectivity

Open the **Command Prompt** on each computer and use the **ping** command to test connectivity with the other computers and the switch.

Example:

- `ping 10.1.1.2`
- `ping 10.1.1.3`
- `ping 10.1.1.4`
- `ping 10.1.1.5`

If replies are received without packet loss, the network has been configured successfully.

Step 7: Share a Folder

1. Create or select a folder to be shared.
2. Right-click the folder and select **Properties**.

3. Open the **Sharing** tab.
4. Click **Share**.
5. Select **Everyone** from the list of users.
6. Click **Share** and then **Done**.

The selected folder is now accessible to all computers on the same LAN.

Step 8: Access the Shared Folder

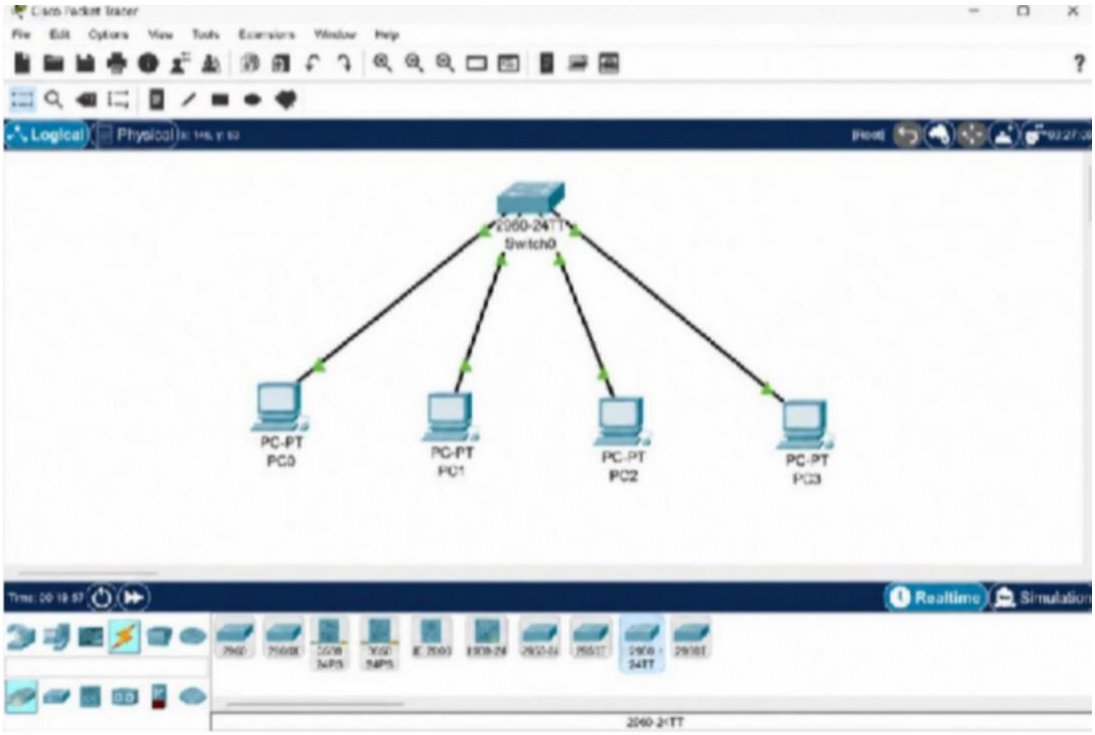
1. Open **File Explorer** on another computer connected to the LAN.
2. In the address bar, type the IP address or computer name of the shared computer. For example:
   - `\\10.1.1.1`
   - `\\PC1`
3. Press **Enter** to view the shared folder.
4. Open the folder and verify that files can be accessed successfully.

Procedure to Set Up a LAN Using Cisco Packet Tracer

Step 1: Design the Network Topology

1. Open **Cisco Packet Tracer**.
2. From the **Network Devices** section, select **Switches** and drag one **2960 Switch** onto the workspace.
3. From the **End Devices** section, drag four **PCs** onto the workspace.
4. Rename the devices as **PC0, PC1, PC2, and PC3**.

Sample Design:



Step 2: Connect the Devices

1. Click **Connections** (Lightning Bolt icon).
2. Select **Copper Straight-Through Cable**.
3. Connect the devices as follows:

Source Device	Interface	Destination Device	Interface
PC0	FastEthernet0	Switch0	FastEthernet0/1
PC1	FastEthernet0	Switch0	FastEthernet0/2
PC2	FastEthernet0	Switch0	FastEthernet0/3
PC3	FastEthernet0	Switch0	FastEthernet0/4

4. Wait until the link LEDs turn **green**, indicating that the devices are successfully connected.

Step 3: Assign IP Addresses to the PCs

For each PC:

1. Click the PC.
2. Select the **Desktop** tab.
3. Click **IP Configuration**.
4. Enter the following IP addresses:

Device	IP Address	Subnet Mask
PC0	10.1.1.1	255.0.0.0
PC1	10.1.1.2	255.0.0.0
PC2	10.1.1.3	255.0.0.0
PC3	10.1.1.4	255.0.0.0

Leave the **Default Gateway** blank because no router is used in this LAN.

Step 4: Verify IP Configuration

On each PC:

1. Open **Desktop**.
2. Select **Command Prompt**.
3. Execute the following command:

```
ipconfig
```

Verify that the configured IP address and subnet mask are displayed correctly.

Step 5: Test Network Connectivity

From **PC0**, open **Command Prompt** and execute:

```
ping 10.1.1.2
```

Similarly, test connectivity with the remaining PCs:

```
ping 10.1.1.3
```

```
ping 10.1.1.4
```

Repeat the ping test from each PC to ensure that all hosts can communicate successfully.

Step 6: Verify Packet Transmission Using Simulation Mode

1. Click **Simulation** mode at the bottom-right corner of Packet Tracer.
2. Select the **Add Simple PDU** tool (Envelope icon).
3. Click **PC0** as the source device.
4. Click **PC3** as the destination device.
5. Click **Auto Capture/Play**.

Observe the ICMP Echo Request and Echo Reply packets travelling through the switch.

Step 7: Configure an FTP Server for File Sharing

Since Packet Tracer does not support Windows folder sharing, use an FTP server to simulate file sharing.

1. Drag one **Server** from **End Devices** into the workspace.
2. Connect the server to the switch using a **Copper Straight-Through Cable**.
3. Configure the server with:

Device	IP Address	Subnet Mask
Server0	10.1.1.5	255.0.0.0

4. Open **Server0**.
5. Select **Services → FTP**.
6. Turn the FTP service **ON**.
7. Create a username and password (for example, **student / student123**).

Step 8: Access the FTP Server

On **PC0**:

1. Open **Desktop → Command Prompt**.
2. Connect to the FTP server using:

```
ftp 10.1.1.5
```

3. Enter the configured username and password.
4. Execute FTP commands such as:

```
dir
get sample.txt
put report.txt
```

Verify that files can be uploaded to and downloaded from the FTP server.

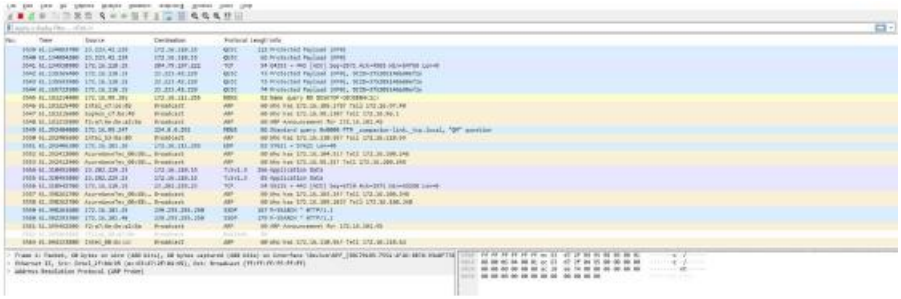
Step 9: Save the Project

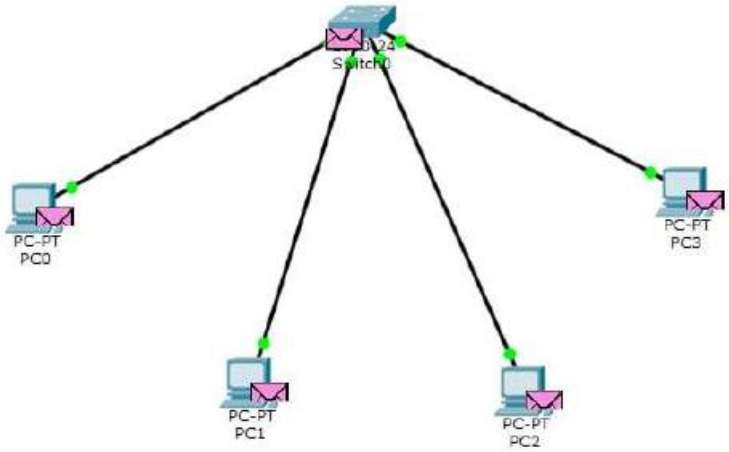
1. Click **File → Save As**.
2. Save the Packet Tracer file with an appropriate name, for example:

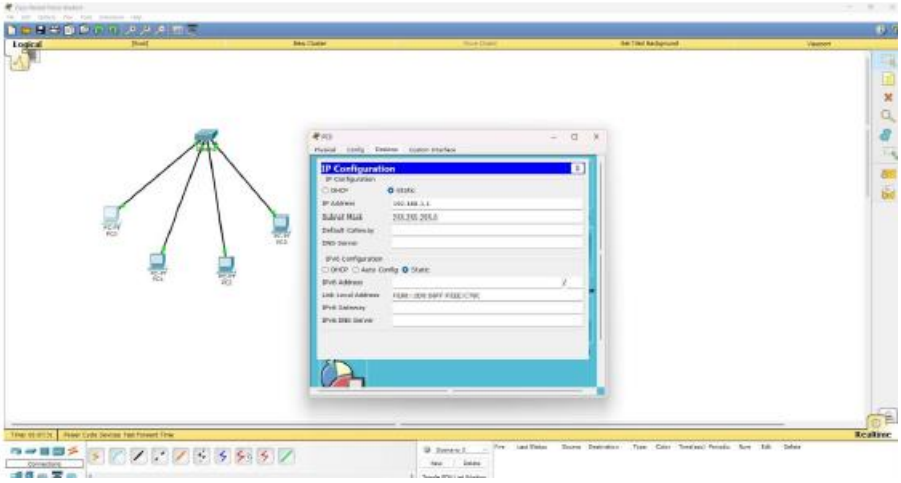
```
LAN_Configuration.pkt
```

The LAN configuration is now complete.

OUTPUT:







RESULT:

Thus, a LAN was configured using a switch and Ethernet cables. IP addresses were assigned, connectivity was verified using ping, and file sharing was performed successfully.

Ex.No.5	Experiments on Packet capture tool: Wireshark
Date	

AIM:

To understand the features of wireshark as a packet capture tool and understand **encapsulation of information at various layers of a Protocol stack**.

WIRESHARK

Wireshark is a free, open-source network packet analyzer used for network troubleshooting, analysis, and security auditing. It lets you capture and interactively browse live data traffic passing through a network interface:

Workdone by Wireshark:

- Capture network traffic
- Decode packet protocols using dissectors
- Define filters – capture and display
- Watch smart statistics
- Analyze problems
- Interactively browse that traffic

Wireshark used for:

- Network administrators: troubleshoot network problems
- Network security engineers: examine security problems
- Developers: debug protocol implementations
- People: learn **network protocol internals**

Getting Wireshark

Wireshark can be downloaded for Windows or macOS from its official website. For Linux or another UNIX-like system, Wireshark will be found in its package repositories. For Ubuntu, Wireshark will be found in the Ubuntu Software Center.

Download Wireshark from the following link

https://www.wireshark.org/download.html

Procedure to Capture Packets Using Wireshark

Step 1: Download and install **Wireshark** on your computer.

Step 2: Open the **Wireshark** application.

Step 3: From the home screen, select the **network interface** (e.g., Wi-Fi or Ethernet) through which your computer is connected to the network.

Step 4: Click the **Start Capturing Packets** button (blue shark fin icon) or double-click the selected interface.

Step 5: Generate network traffic by opening a website, sending a ping, or browsing the Internet.

Step 6: Observe the captured packets appearing in real time in the Packet List pane.

Step 7: To view only specific packets, enter a **display filter** (e.g., `http`, `tcp`, `udp`, `icmp`, `dns`) in the filter bar and press **Enter**.

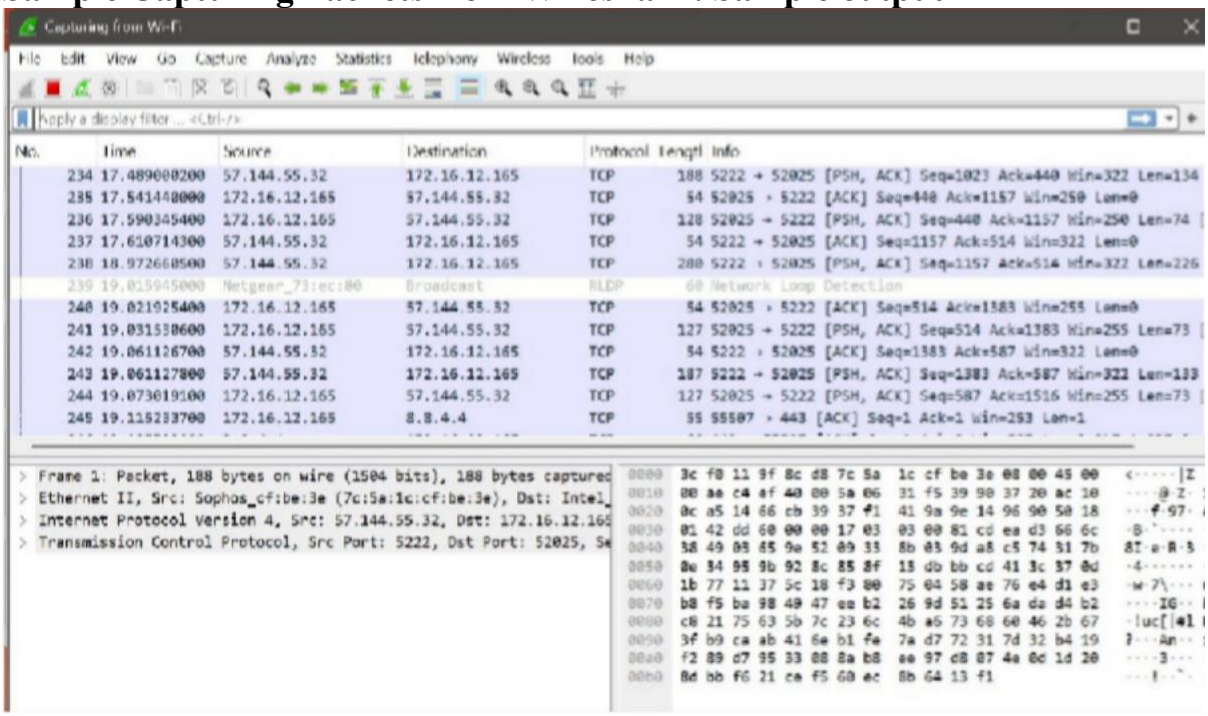
Step 8: Click on any packet to view its detailed information in the **Packet Details** pane.

Step 9: Expand the protocol layers (Ethernet, IP, TCP/UDP, etc.) to examine packet contents.

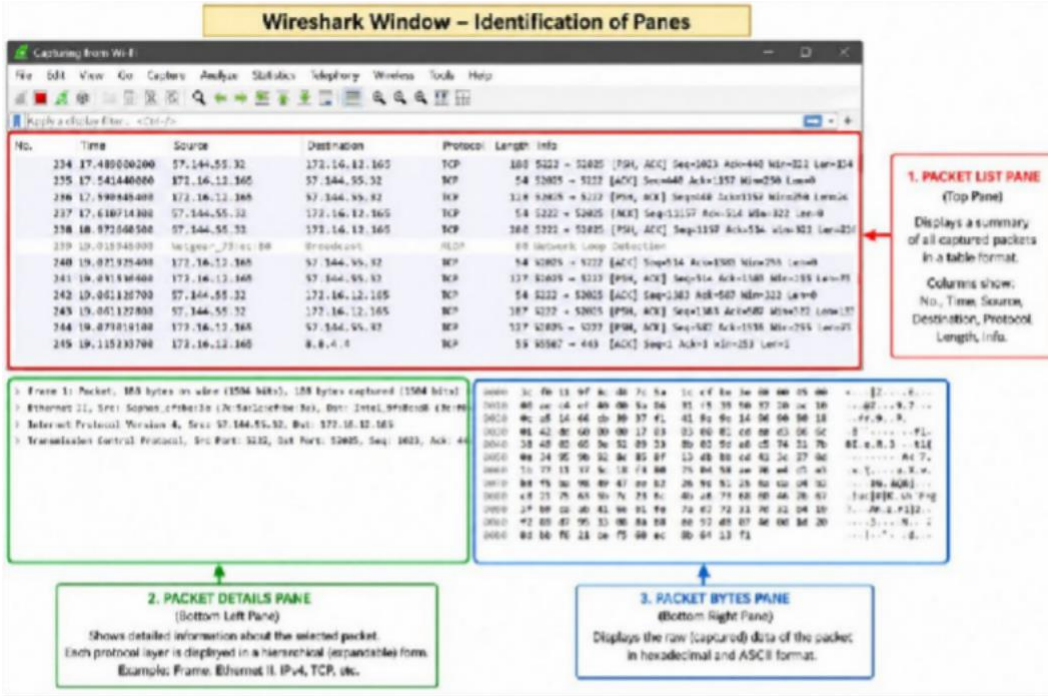
Step 10: Click the **Stop** button (red square) to stop packet capture.

Step 11: To save the captured packets, go to **File → Save As**, choose a location, and save the file in **.pcapng** (or **.pcap**) format.

Sample Capturing Packets from Wireshark: Sample output



Different Panes in Wireshark:



1. Packet List Pane (Top Pane)

- Displays a summary of all captured packets.
- Each row represents one captured packet.
- Shows details such as **Packet Number, Time, Source, Destination, Protocol, Length,** and **Info**.

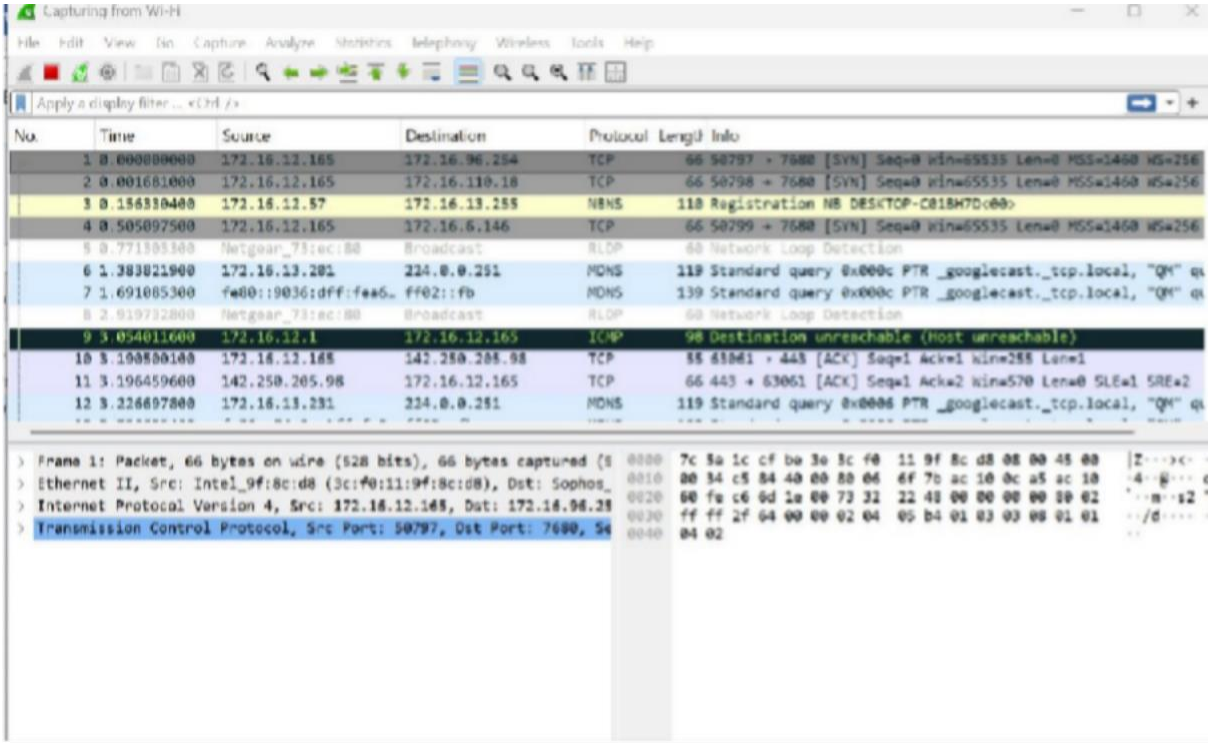
2. Packet Details Pane (Middle Pane)

- Displays detailed information about the selected packet.
- Information is organized into protocol layers such as **Frame, Ethernet, IP, TCP/UDP**, etc.
- Each protocol layer can be expanded to view more details.

3. Packet Bytes Pane (Bottom Pane)

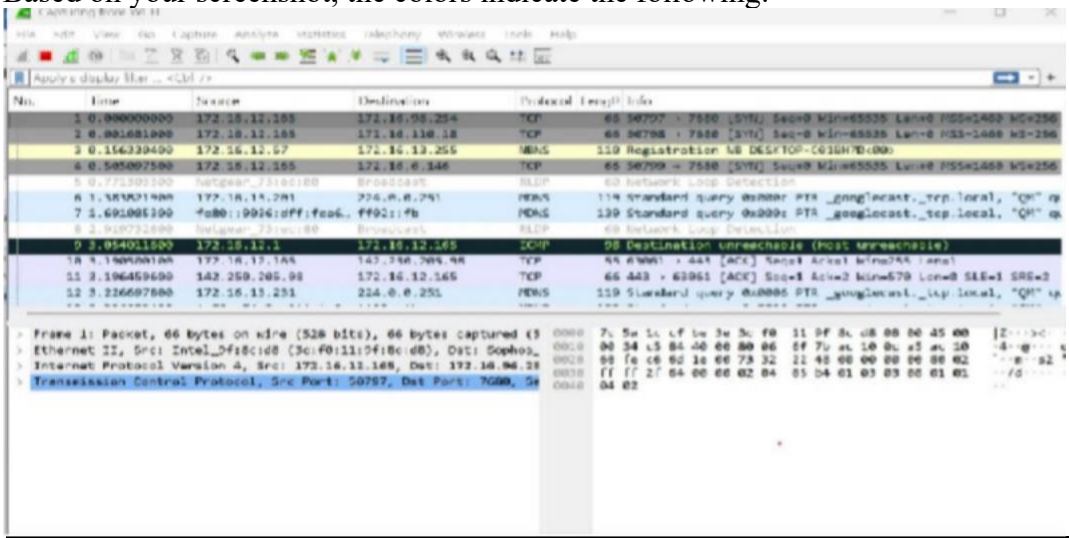
- Displays the raw data of the selected packet.
- Data is shown in **hexadecimal** format on the left.
- The corresponding **ASCII** representation is shown on the right.

Note: These three panes work together to help users analyze captured network packets efficiently in Wireshark.



Color Coding:

In Wireshark, **packet colors** help you quickly identify different types of network traffic. Based on your screenshot, the colors indicate the following:

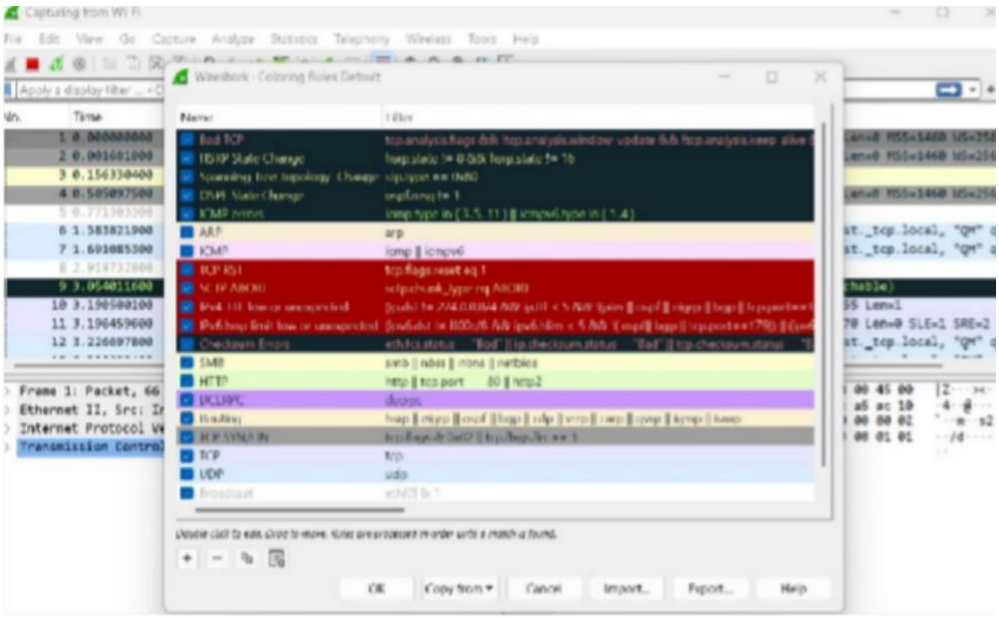


Color	Protocol/Traffic	Description
Light Gray	TCP	Indicates normal TCP packets such as SYN and ACK packets.
Light Yellow	NBNS (NetBIOS Name Service)	Used for NetBIOS name registration and name resolution on a local network.
White	RLDP (Network Loop Detection)	Represents Network Loop Detection Protocol packets used by network devices.
Light Blue	MDNS (Multicast DNS)	Used for local network service discovery (e.g., Chromecast, printers, Apple devices).
Dark Green / Black with Green Text	ICMP	Indicates Internet Control Message Protocol packets such as **Destination Unreachable**, **Ping Request**, or **Ping Reply**.
Light Purple /	TCP	Represents another category of TCP packets (e.g.,

Color	Protocol/Traffic	Description
Lavender		ACK or established connection traffic) based on Wireshark's default coloring rules.

To view exactly what the color codes mean, click **View > Coloring Rules.** You can also customize and modify the coloring rules from here, if you like.

Sample Output:



Sample Captures:

- If there is no interesting network traffic on your own network, you can use **sample capture files**.
- Download a sample capture file and open it by selecting **File → Open** in Wireshark.
- Browse to the downloaded file and click **Open** to analyze the captured packets.
- You can also save your own captured packets for future analysis by selecting **File → Save**.
- Wireshark saves capture files in **.pcap** or **.pcapng** format.

Simple Procedure:

1. Download a sample capture file.
2. Open Wireshark.
3. Click **File → Open**.
4. Select the sample capture file.
5. Analyze the captured packets.
6. To save your own capture, click **File → Save**.

You can also save your own captures in Wireshark and open them later. Click **File > Save** to save your captured packets

Filtering Packets:

- Close unnecessary applications before capturing packets.
- This reduces unwanted network traffic.
- Capture only the traffic related to the program you want to analyze.
- Without filters, thousands of packets may be captured.
- Filters help narrow down the packet list.
- This makes packet analysis faster and easier.
- Wireshark provides **Capture Filters** and **Display Filters** to simplify traffic analysis.

Sample filter applied for TCP:

Filtered Packet List

No.	Time	Source	Destination	Protocol	Length	Info
1	0.001	192.168.1.10	142.250.183.78	TCP	74	50234 → 443 [SYN]
2	0.005	142.250.183.78	192.168.1.10	TCP	74	443 → 50234 [SYN, ACK]
3	0.006	192.168.1.10	142.250.183.78	TCP	66	50234 → 443 [ACK]
4	0.020	192.168.1.10	142.250.183.78	TCP	1514	HTTP GET Request
5	0.055	142.250.183.78	192.168.1.10	TCP	1514	HTTP 200 OK

Display Filters in Wireshark:

- Click **Analyze → Display Filters** to open the list of available display filters.
- Choose a **predefined filter** from the default list.
- You can **create your own custom filters**.
- Save custom filters for **future use**.
- Display filters make it easier to find specific packets.
- For advanced filter syntax, refer to the **official Wireshark Display Filter documentation**.

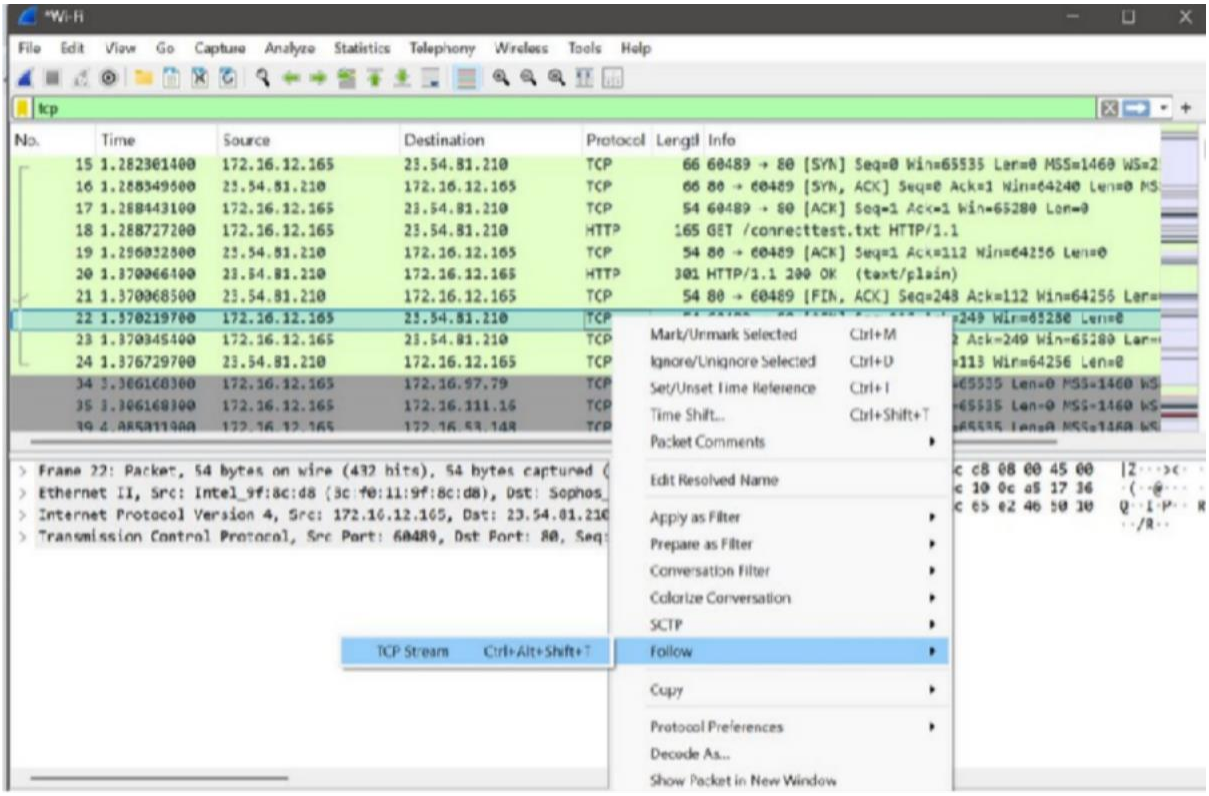
Example: Create and Save a Custom Filter

Suppose you frequently analyze **DNS packets**.

1. Click **Analyze → Display Filters**.
2. Click the **+ (Add)** button.
3. Enter:
   - **Filter Name:** `DNS Packets`
   - **Filter Expression:** `dns`
4. Click **OK**.

Follow TCP Stream in Wireshark

1. Right-click on a **TCP packet**.
2. Select **Follow → TCP Stream**.
3. Wireshark displays the **complete conversation** between the client and the server.
4. This helps you understand the **data exchanged** during the communication.
5. You can also choose **Follow → UDP Stream** or other available protocol streams, if supported.
6. This feature is useful for analyzing web requests, application data, and troubleshooting network communication.



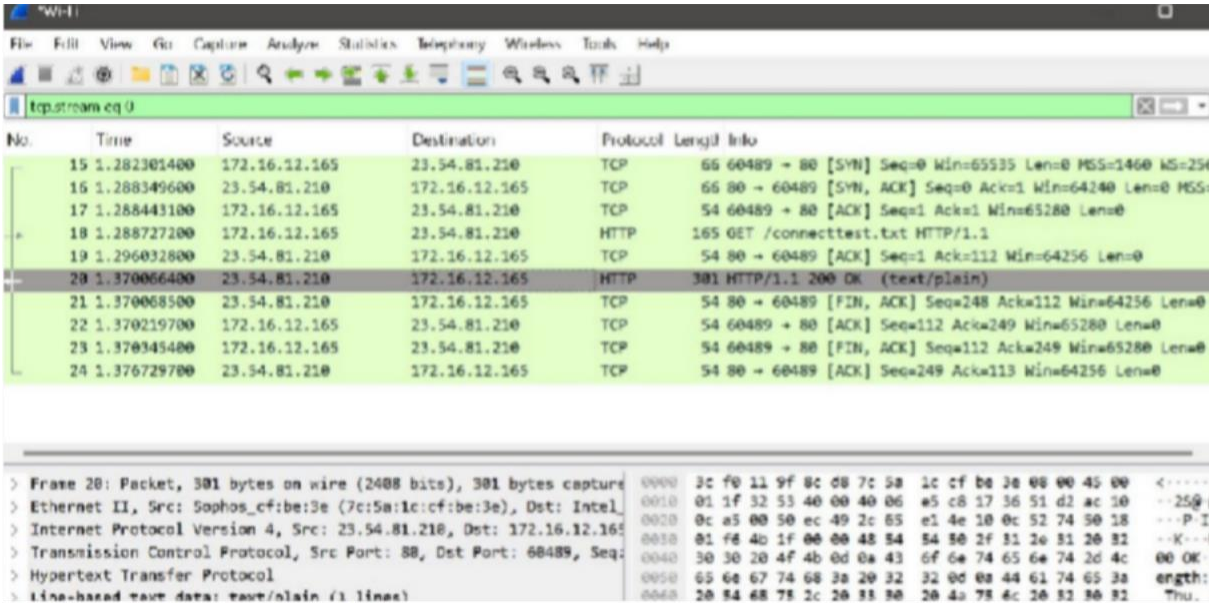
Sample Output:

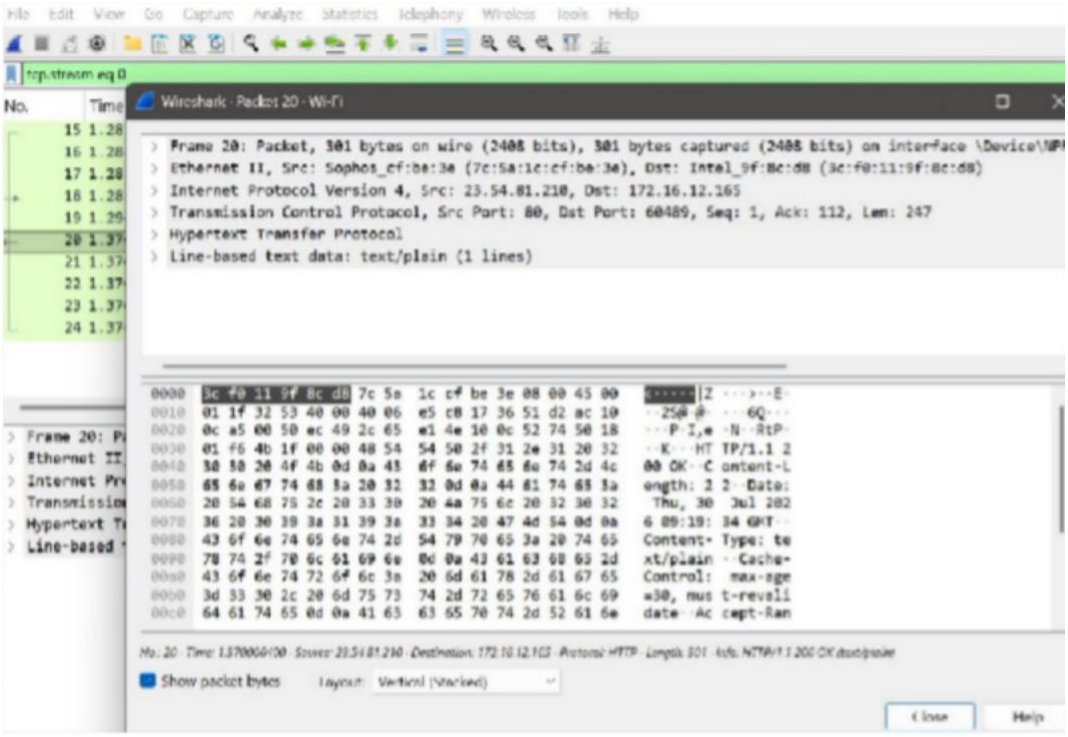
No.	Source	Destination	Protocol	Info
35	192.168.1.10	93.184.216.34	TCP	50321 → 80 [SYN]
36	93.184.216.34	192.168.1.10	TCP	80 → 50321 [SYN, ACK]
37	192.168.1.10	93.184.216.34	TCP	50321 → 80 [ACK]
38	192.168.1.10	93.184.216.34	HTTP	GET /index.html
39	93.184.216.34	192.168.1.10	HTTP	HTTP/1.1 200 OK

Inspecting Packets:

- Click a **packet** in the Packet List pane.
- The packet details are displayed in the **Packet Details** pane.
- Expand each protocol layer (Ethernet, IP, TCP/UDP, etc.) to view more information.
- View the raw packet data in the **Packet Bytes** pane.
- Inspecting packets helps analyze **source and destination addresses, protocol information, ports, and packet contents**.

Sample Output:





Flow Graph in Wireshark

Definition:
A **Flow Graph** visually represents the sequence of communication between different devices (hosts) in a network. It makes it easier to understand how packets are exchanged.

How to Open Flow Graph

1. Open a packet capture (.pcap) file.
2. Click **Statistics → Flow Graph**.
3. A Flow Graph window appears showing the communication between hosts.

What the Graph Shows

- **Vertical lines** represent different network devices (hosts).

- **Horizontal arrows** represent packets sent between hosts.
- **Time** is shown from **top to bottom**.
- Each arrow is labeled with the **protocol** and **packet information**.

Example:

Yes. Below is a **sample Flow Graph output** similar to what you would see in Wireshark after selecting **Statistics → Flow Graph**.

Sample Output: Flow Graph

```
Time        Client (192.168.1.10)          Server (93.184.216.34)

0.000       |--------------------------------------------->|  DNS Query
0.015       |<---------------------------------------------|  DNS Response

0.030       |--------------------------------------------->|  TCP SYN
0.035       |<---------------------------------------------|  TCP SYN-ACK
0.040       |--------------------------------------------->|  TCP ACK

0.050       |--------------------------------------------->|   HTTP GET /index.html
0.120       |<---------------------------------------------|  HTTP/1.1 200 OK
0.125       |<---------------------------------------------|  HTML Page Data

0.150       |--------------------------------------------->|  TCP FIN
0.155       |<---------------------------------------------|  TCP ACK
0.160       |<---------------------------------------------|  TCP FIN
0.165       |--------------------------------------------->|  TCP ACK
```

Wireshark Capture Options:

- ➢ Go to capture ☾ option
- ➢ Select stop capture automatically after 100 packets.
- ➢ Then click Start capture.

Step 1: Select the Active Interface

In your screenshot, choose the interface that is currently connected to the network.

- **If you are using Wi-Fi**, click **Wi-Fi**.
- **If you are using a LAN cable**, click **Ethernet**.

Tip: The active interface usually shows a moving graph or increasing traffic. In your screenshot, **Wi-Fi** is likely the correct choice if you are connected wirelessly.

Step 2: Go to the Options Tab

Click the **Options** tab at the top.

You will see settings similar to this:

☑ Stop capture automatically after

Step 3: Set the Packet Limit

Check the option:

☑ Stop capture automatically after

Select:

100 Packets

Creating a Filter to display only DHCP packets and inspect the packets.

Procedure

1. Open **Wireshark**.
2. Select the **Ethernet (Local Area Connection)** or **Wi-Fi** interface.
3. Click **Capture → Options**.
4. Select **Stop capture automatically after 100 packets**.
5. Click **Start** to begin packet capture.

6. Generate DHCP traffic by disconnecting and reconnecting the network, or by running:

7.
```
ipconfig /release
ipconfig /renew
```

8. After the capture stops, type the following in the **Display Filter** bar:

   ```
   dhcp
   ```

   (If `dhcp` does not work, use `bootp`.)

9. Press **Enter** or click **Apply** to display only DHCP packets.
10. Click any DHCP packet to inspect its details.
11. Click **File → Save As**, enter a filename, and save the captured packets as a **.pcapng** file.

Create a Filter to display only IP/ICMP packets and inspect the packets. Procedure

Step 1: Check if ping is working

1. Press **Windows + R**.
2. Type **cmd** and press **Enter**.

3. Type:

   ```
   ping 8.8.8.8
   ```

4. Press **Enter**.

What do you see?

- If you see:

  ```
  Reply from 8.8.8.8...
  ```

  then the ping is working.

- If you see:

  ```
  Request timed out
  ```

or

```
Ping request could not find host...
```

tell me exactly what message appears.

Step 2: Check whether Wireshark is capturing

When you click **Start Capture**, do you see the packet count increasing continuously?

- **Yes** → Wireshark is capturing.
- **No** → The wrong network interface is selected.

Step 3: Verify the filter

After capturing packets, clear any existing filter and type:

```
icmp
```

If the filter bar turns **green**, the filter is valid.

If it stays green but shows **0 packets**, then no ICMP packets were captured.

Step 4: Easier alternative

Instead of using `ping`, try this:

1. Start Wireshark.
2. Open **Google Chrome**.
3. Visit **www.google.com**.
4. Stop the capture.
5. In the Display Filter, type:

   ```
   dns
   ```

   or

   ```
   tcp
   ```

6. Press **Enter**.

These filters almost always show packets because web browsing generates DNS and TCP traffic.

Sample Output (ICMP Filter)

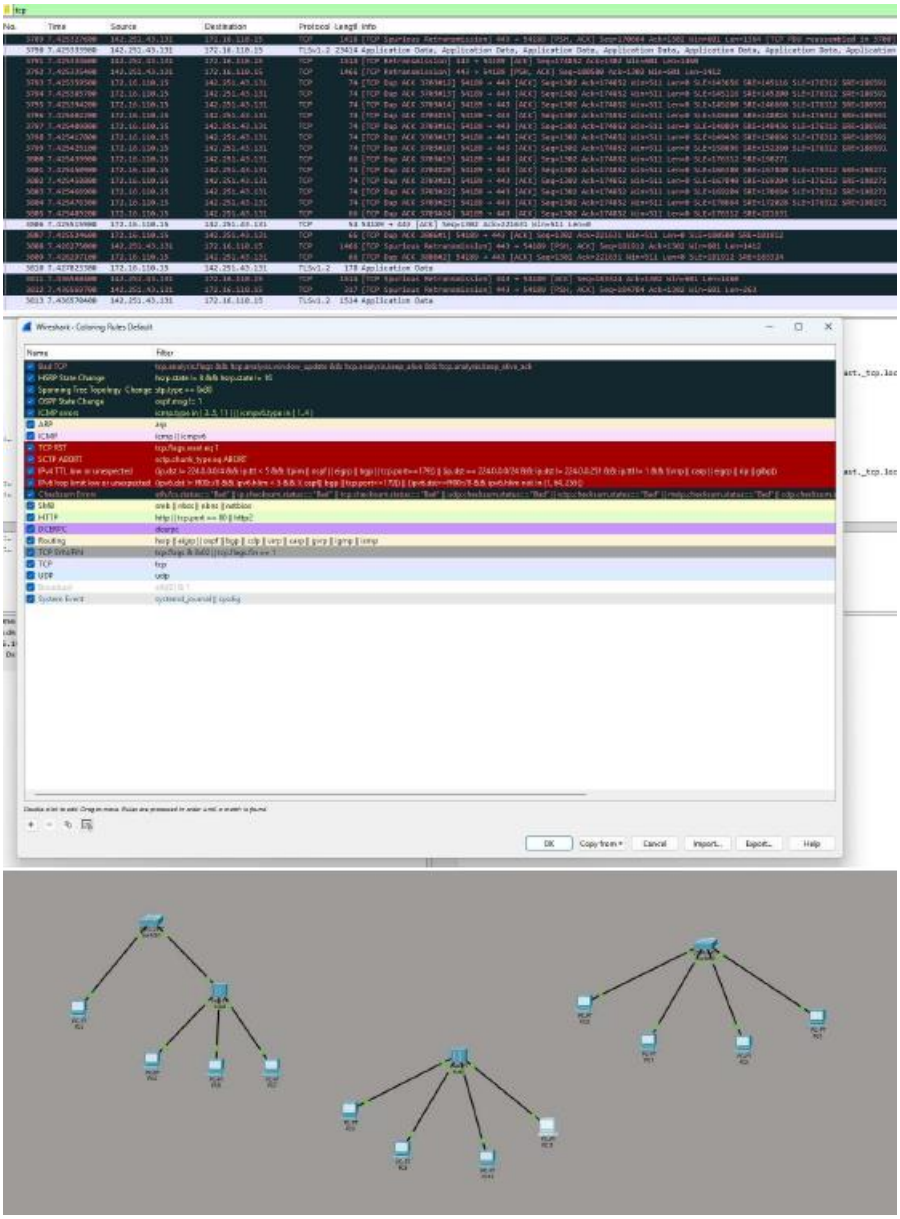
Display Filter:

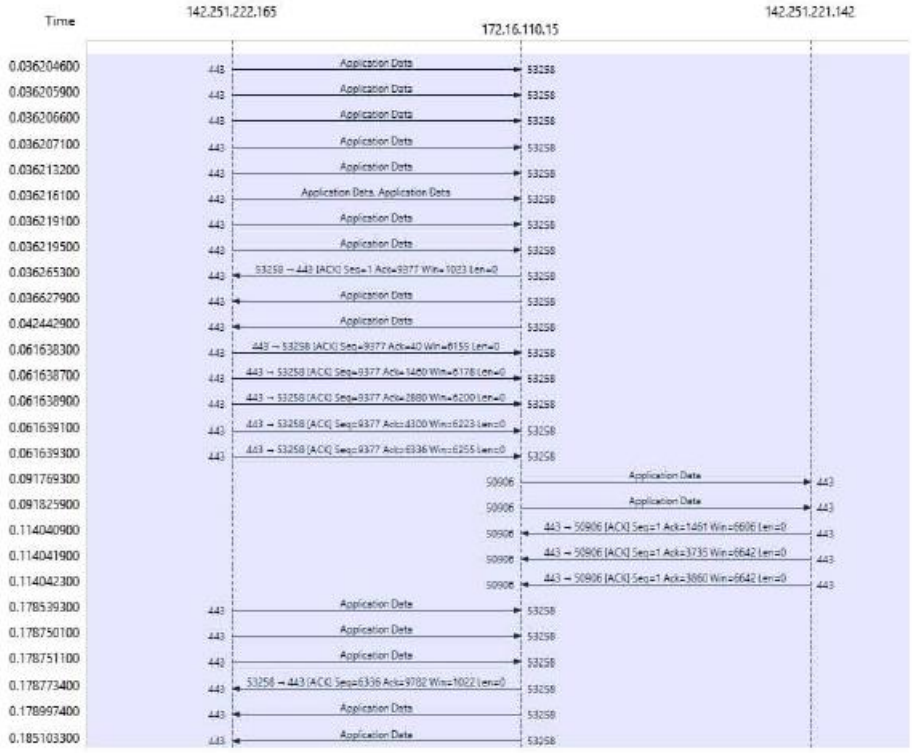
```
icmp
```

Packet List

No.	Time	Source	Destination	Protocol	Length	Info
15	2.345	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
16	2.356	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply
17	3.347	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
18	3.359	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply
19	4.351	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
20	4.362	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) repl

OUTPUT:





RESULT:

Thus, network packets were captured and analyzed using Wireshark, and the encapsulation of data at different protocol layers was studied.

Ex.No.6	**Building anonymous FTP Scanner using ftplib module**
DATE:	

AIM:

To build an anonymous ftp scanner using ftplib module in python.

Introduction:

FTP Scanner:

A tool that automatically scans IPs/hosts for active FTP servers. It tests connectivity and credentials, reporting successful logins and vulnerabilities.

ftplib:

A built-in Python library for working with the File Transfer Protocol (FTP). It allows connecting to FTP servers, logging in, listing directories, and transferring files.

ALGORITHM:

1. Start.
2. Import the `ftplib` module.
3. Specify the FTP server hostname.
4. Connect to the FTP server.
5. Log in using the username **anonymous** and password **anonymous**.
6. If the login is successful:
    - Display the login status.
    - Display the server's welcome message.
    - Display the directory listing.
7. Close the FTP connection.
8. If the login fails, display an error message.
9. Stop.

PROGRAM:

```
import ftplib
def anonymous_login(hostname):
    try:
        # Connect to the FTP server
        ftp = ftplib.FTP(hostname)
        # Login using anonymous credentials
        response = ftp.login('anonymous', 'anonymous')
        print(response)
        # Check whether anonymous login is successful
        if "230" in response:
            print("\n[*] FTP Anonymous Login Succeeded.")
            print("Server:", hostname)
            # Display welcome message
            print("\nWelcome Message:")
            print(ftp.getwelcome())
            # Display directory listing
            print("\nDirectory Listing:")
            ftp.dir()
        # Close the FTP connection
        ftp.quit()
    except Exception as e:
        print("Error:", e)
        print("\n[-] FTP Anonymous Login Failed.")
# Main Program
hostname = "ftp.be.debian.org"
anonymous_login(hostname)
```

SAMPLE OUTPUT:

```
230 Login successful.
```

[*] FTP Anonymous Login Succeeded.

Server: ftp.be.debian.org

Welcome Message:

220 FTP Server Ready

Directory Listing:

drwxr-xr-x 2 ftp ftp 4096 Jan 10 2026 debian

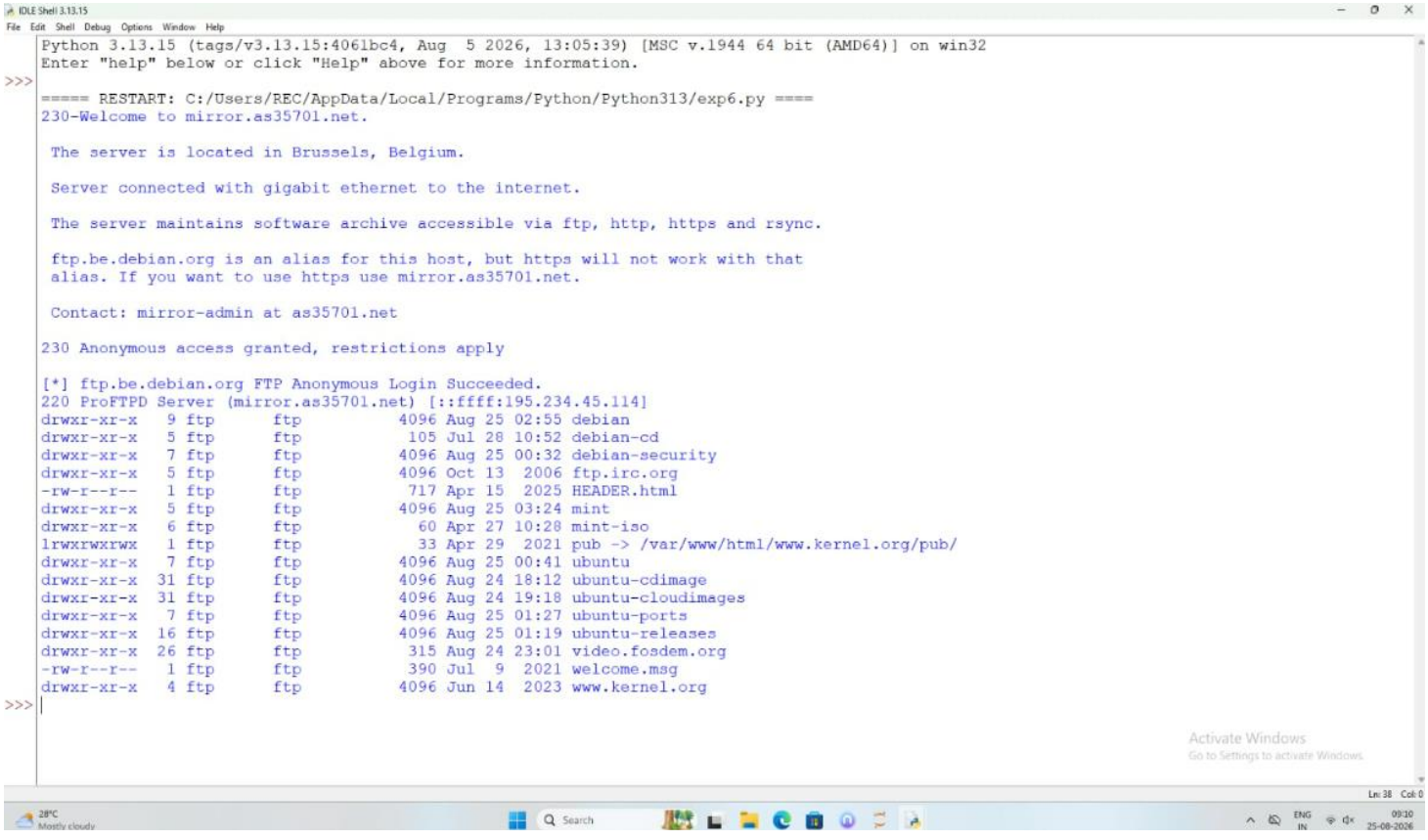
drwxr-xr-x 2 ftp ftp 4096 Jan 10 2026 cdimage

drwxr-xr-x 2 ftp ftp 4096 Jan 10 2026 project

-rw-r--r-- 1 ftp ftp 512 README.txt

221 Goodbye.

OUTPUT:



RESULT:

Thus an anonymous ftp scanner using ftplib module was built and executed successfully.

Ex.No.7	Develop a simple calculator using XMLRPC
DATE	

AIM:

To develop a simple calculator using XMLRPC using Python.

Introduction: XML-RPC:

XML-RPC is a protocol that uses XML to encode remote procedure calls (RPC) and HTTP as a transport mechanism, allowing a client to call functions on a remote server over a network.

ALGORITHM:

CLIENT:

1. Start the program.
2. Import the `xmlrpc.client` module.
3. Create a connection to the XML-RPC server using `ServerProxy`.
4. Display the calculator menu:
   - Addition
   - Subtraction
   - Multiplication
   - Division
   - Exit
5. Read the user's choice.
6. If the user selects **Exit**, terminate the program.
7. Otherwise, read two input numbers.
8. Send the selected operation and the two numbers to the server.
9. Receive the result from the server.
10. Display the result.
11. Repeat Steps 4–10 until the user chooses **Exit**.
12. End the program.

SERVER:

1. Start the program.
2. Import the SimpleXMLRPCServer module.
3. Define the calculator functions:
   Addition
   Subtraction
   Multiplication
   Division (check for division by zero).
4. Create an XML-RPC server on localhost with port 8000.
5. Register all the calculator functions with the server.
6. Display the message **"XML-RPC Server is Running on Port 8000..."**.
7. Wait for client requests.
8. When a client sends a request:
9. Execute the requested arithmetic operation.
10. Return the result to the client.
11. Continue serving requests until the server is manually stopped.

12. End the program.

PROGRAM CODE:

SERVER:

```
from xmlrpc.server import SimpleXMLRPCServer
# Calculator Functions
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

def mul(a, b):
    return a * b

def div(a, b):
    if b == 0:
        return "Error! Division by Zero"
    return a / b

# Create XML-RPC Server
server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC Server is Running on Port 8000...")

# Register Functions
server.register_function(add, "add")
server.register_function(sub, "sub")
server.register_function(mul, "mul")
server.register_function(div, "div")

# Keep Server Running
server.serve_forever()
```

CLIENT:

```
import xmlrpc.client
# Connect to Server
proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
while True:
    print("\n------ XML-RPC Calculator------ ")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Multiplication")
    print("4. Division")
    print("5. Exit")

    choice = int(input("Enter your Choice: "))

    if choice == 5:
        print("Calculator Closed.")
        break

    a = float(input("Enter First Number: "))
    b = float(input("Enter Second Number: "))

    if choice == 1:
        print("Result =", proxy.add(a, b))

    elif choice == 2:
        print("Result =", proxy.sub(a, b))

    elif choice == 3:
        print("Result =", proxy.mul(a, b))

    elif choice == 4:
        print("Result =", proxy.div(a, b))

    else:
        print("Invalid Choice")
```

SAMPLE OUTPUT:

SERVER:

```
XML-RPC Server is Running on Port 8000...

127.0.0.1 - - [09/Jul/2026 11:05:10] "POST /RPC2 HTTP/1.1" 200 -
127.0.0.1 - - [09/Jul/2026 11:05:24] "POST /RPC2 HTTP/1.1" 200 -
127.0.0.1 - - [09/Jul/2026 11:05:38] "POST /RPC2 HTTP/1.1" 200 -
127.0.0.1 - - [09/Jul/2026 11:05:52] "POST /RPC2 HTTP/1.1" 200 -
```

CLIENT:

```
------ XML-RPC Calculator ------
1. Addition
2. Subtraction
```

3. Multiplication
4. Division
5. Exit

Enter your Choice: 1
Enter First Number: 15
Enter Second Number: 10
Result = 25.0

------ XML-RPC Calculator ------
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your Choice: 2
Enter First Number: 15
Enter Second Number: 10
Result = 5.0

------ XML-RPC Calculator ------
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your Choice: 3
Enter First Number: 15
Enter Second Number: 10
Result = 150.0

------ XML-RPC Calculator ------
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your Choice: 4
Enter First Number: 15
Enter Second Number: 10
Result = 1.5

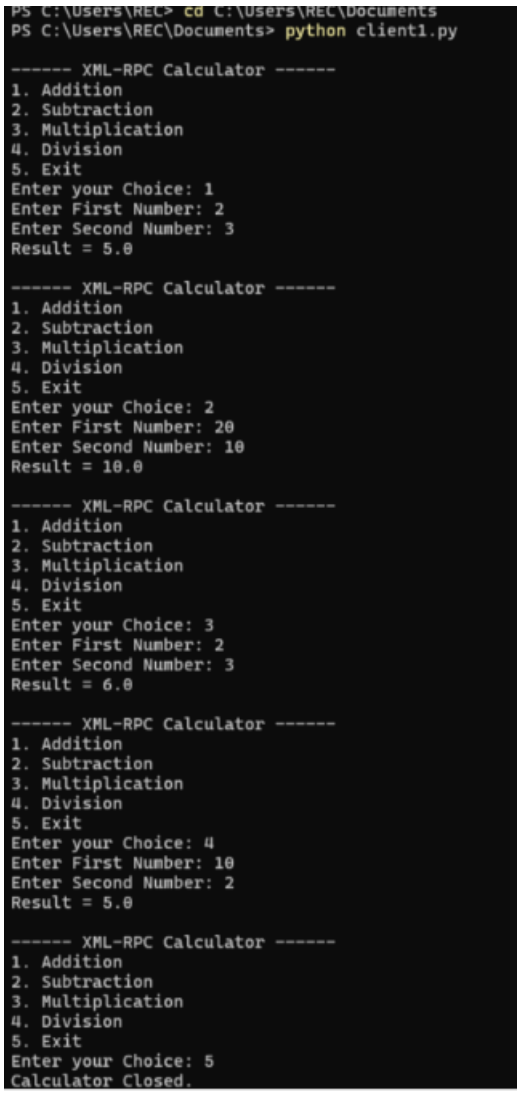
------ XML-RPC Calculator ------
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your Choice: 4
Enter First Number: 15
Enter Second Number: 0
Result = Error! Division by Zero

------ XML-RPC Calculator ------
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your Choice: 5
Calculator Closed.

OUTPUT:



RESULT:

Thus, a simple calculator was developed and executed successfully using XML-RPC in Python.

Ex.No.8	Examine Network Address Translation (NAT) using Cisco Packet Tracer
Date:	

AIM

To configure and examine **Network Address Translation (NAT)** using **Cisco Packet Tracer** and verify communication between a private network and a public network.

Requirements

- Cisco Packet Tracer
- 1 Router (Cisco 1941)
- 2 Switches (Cisco 2960)
- 2 PCs
- Copper Straight-Through cables

Network Topology

Private Network Public Network

PC0 -------- Switch0 -------- Router -------- Switch1 --------- PC1
192.168.1.10 NAT Router 203.0.113.10
G0/0 Public PC
192.168.1.1
|
G0/1
203.0.113.1

IP Addressing

Device	Interface	IP Address	Subnet Mask
PC0	FastEthernet0	192.168.1.10	255.255.255.0
Router	G0/0	192.168.1.1	255.255.255.0
Router	G0/1	203.0.113.1	255.255.255.0
PC1	FastEthernet0	203.0.113.10	255.255.255.0

PROCEDURE

Step 1: Create the topology

1. Open **Cisco Packet Tracer**.
2. Place:
   - Two PCs.
   - Two 2960 switches.
   - One 1941 router.
3. Connect:
   - PC0 → Switch0.
   - Switch0 → Router G0/0.
   - Router G0/1 → Switch1.
   - Switch1 → PC1.
4. Use **Copper Straight-Through** cables.

Step 2: Configure PC0

Go to:

PC0 → Desktop → IP Configuration

Enter:

IP Address : 192.168.1.10
Subnet Mask : 255.255.255.0
Default Gateway : 192.168.1.1
Step 3: Configure PC1

Go to:

PC1 → Desktop → IP Configuration

Enter:

IP Address : 203.0.113.10
Subnet Mask : 255.255.255.0
Default Gateway : 203.0.113.1
Step 4: Configure the Router

Open:

Router → CLI

Configure the inside interface:

```
Router> enable
Router# configure terminal

Router(config)# interface gigabitEthernet 0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# ip nat inside
Router(config-if)# no shutdown
Router(config-if)# exit
```

Configure the outside interface:

```
Router(config)# interface gigabitEthernet 0/1
Router(config-if)# ip address 203.0.113.1 255.255.255.0
Router(config-if)# ip nat outside
Router(config-if)# no shutdown
Router(config-if)# exit
```

Step 5: Configure NAT

Create an access list to identify the private network:

```
Router(config)# access-list 1 permit 192.168.1.0 0.0.0.255
```

Configure **PAT (NAT Overload)**:

Router(config)# ip nat inside source list 1 interface gigabitEthernet 0/1 overload

This allows multiple private IP addresses to share the router's outside IP address.

Step 6: Verify the Router Configuration

Execute:

Router# show ip interface brief

Expected output:

Interface	IP-Address	Status
GigabitEthernet0/0	192.168.1.1	up
GigabitEthernet0/1	203.0.113.1	up

Step 7: Test Connectivity

From **PC0**, open:

Desktop → Command Prompt

Enter:

ping 192.168.1.1

Expected:

Reply from 192.168.1.1: bytes=32 time<1ms TTL=255

Now test the public-side PC:

ping 203.0.113.10

Expected:

Reply from 203.0.113.10: bytes=32 time<1ms TTL=127

Step 8: Verify NAT Translation

Generate traffic from PC0 by pinging PC1:

PC0> ping 203.0.113.10

Then, on the router, enter:

Router# show ip nat translations

Sample output:

Pro	Inside global	Inside local	Outside local	Outside global

icmp 203.0.113.1:1 192.168.1.10:1 203.0.113.10:1 203.0.113.10:1

This shows that the **private IP address 192.168.1.10** has been translated to the router's **public IP address 203.0.113.1**.

Step 9: Verify NAT Statistics

Enter:

Router# show ip nat statistics

Sample output:

Total active translations: 1

Inside interfaces:
GigabitEthernet0/0

Outside interfaces:
GigabitEthernet0/1

Hits: 5
Misses: 0

The exact numbers may vary depending on the number of packets generated.

SAMPLE OUTPUT

PC0 pinging PC1:

PC> ping 203.0.113.10

Pinging 203.0.113.10 with 32 bytes of data:

Reply from 203.0.113.10: bytes=32 time<1ms TTL=127
Reply from 203.0.113.10: bytes=32 time<1ms TTL=127
Reply from 203.0.113.10: bytes=32 time<1ms TTL=127
Reply from 203.0.113.10: bytes=32 time<1ms TTL=127

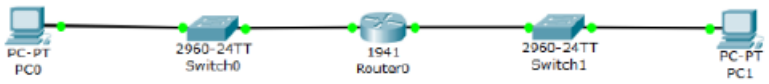
Ping statistics:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

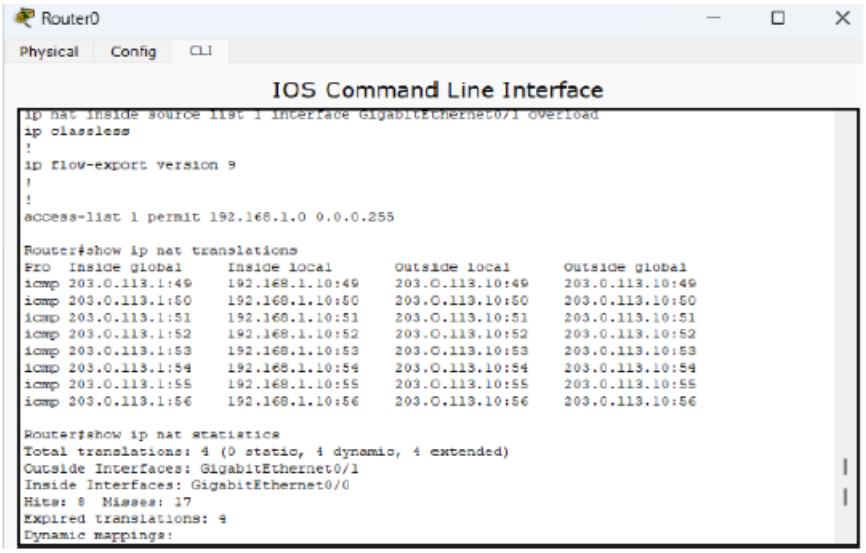
NAT Translation Table:

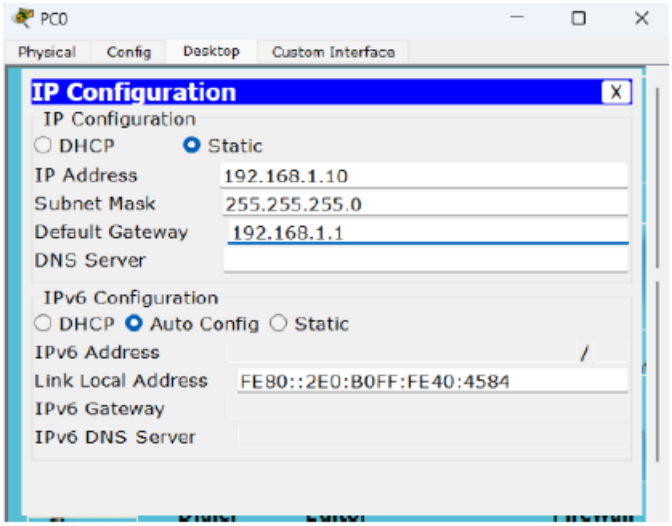
Router# show ip nat translations

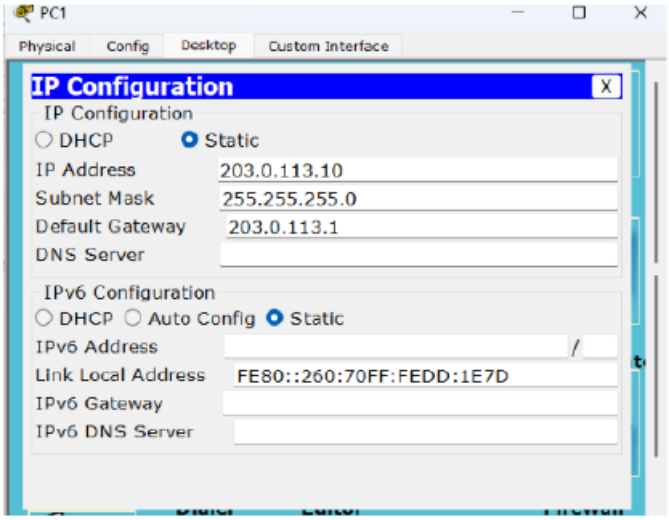
Pro	Inside global	Inside local	Outside local	Outside global
icmp	203.0.113.1:1	192.168.1.10:1	203.0.113.10:1	203.0.113.10:1

OUTPUT:


PC-PT
PC0
2960-24TT
Switch0
1941
Router0
2960-24TT
Switch1
PC-PT
PC1


Router0
Physical
Config
CLI
IOS Command Line Interface
ip nat inside source list 1 interface GigabitEthernet0/1 overload
ip classless
!
ip flow-export version 9
!
!
access-list 1 permit 192.168.1.0 0.0.0.255
Router#show ip nat translations
Pro Inside global Inside local Outside local Outside global
icmp 203.0.113.1:49 192.168.1.10:49 203.0.113.10:49 203.0.113.10:49
icmp 203.0.113.1:50 192.168.1.10:50 203.0.113.10:50 203.0.113.10:50
icmp 203.0.113.1:51 192.168.1.10:51 203.0.113.10:51 203.0.113.10:51
icmp 203.0.113.1:52 192.168.1.10:52 203.0.113.10:52 203.0.113.10:52
icmp 203.0.113.1:53 192.168.1.10:53 203.0.113.10:53 203.0.113.10:53
icmp 203.0.113.1:54 192.168.1.10:54 203.0.113.10:54 203.0.113.10:54
icmp 203.0.113.1:55 192.168.1.10:55 203.0.113.10:55 203.0.113.10:55
icmp 203.0.113.1:56 192.168.1.10:56 203.0.113.10:56 203.0.113.10:56
Router#show ip nat statistics
Total translations: 4 (0 static, 4 dynamic, 4 extended)
Outside Interfaces: GigabitEthernet0/1
Inside Interfaces: GigabitEthernet0/0
Hits: 8 Misses: 17
Expired translations: 4
Dynamic mappings:





RESULT

Thus, **Network Address Translation (NAT)** was successfully configured and examined using **Cisco Packet Tracer**. The private IP address **192.168.1.10** was translated into the public IP address **203.0.113.1**, and successful communication with the public network was verified.

Ex.No.9	Develop a program to create reverse shell using TCP sockets
DATE:	

AIM:

To write a program to create reverse shell using TCP sockets

ALGORITHM:

Server:
Algorithm: TCP Socket Server

1. Start the program.
2. Import the required modules.
3. Create a TCP socket.
4. Bind the socket to the host address and port number.
5. Listen for incoming client connections.
6. Accept the client connection.
7. Receive the command from the client.
8. Check the received command:
    - If **hello**, send a greeting message.
    - If **time**, send the current time.
    - If **date**, send the current date.
    - If **exit**, send a closing message and terminate the connection.
    - Otherwise, send **"Invalid Command"**.
9. Repeat Steps 7 and 8 until the client exits.
10. Close the client connection and the server socket.
11. Stop the program.

Client:

Algorithm: TCP Socket Client

1. Start the program.
2. Import the socket module.
3. Define the **HOST** address and **PORT** number.
4. Create a TCP socket.
5. Connect to the server using the specified HOST and PORT.
6. Display the list of available commands.
7. Repeat the following steps until the user enters **exit**:
    - Read a command from the user.
    - Send the command to the server.
    - Receive the response from the server.
    - Display the server's response.
    - If the command is **exit**, terminate the loop.
8. Close the client socket.
9. Stop the program.

PROGRAM:

SERVER:

```
import socket
import datetime
HOST = "127.0.0.1"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print("Server is listening on", HOST, PORT)
conn, addr = server.accept()
print("Connected by:", addr)
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("Client Command:", data)
    if data.lower() == "hello":
        response = "Hello from Server!"
    elif data.lower() == "time":
        response = datetime.datetime.now().strftime("%H:%M:%S")
    elif data.lower() == "date":
        response = datetime.datetime.now().strftime("%d-%m-%Y")
    elif data.lower() == "exit":
        response = "Connection Closed"
        conn.send(response.encode())
        break
    else:
        response = "Invalid Command"
    conn.send(response.encode())
conn.close()
server.close()
```

CLIENT:

```
import socket
HOST = "127.0.0.1"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
print("Connected Successfully")
print("Available Commands")
print("1. hello")
print("2. time")
print("3. date")
print("4. exit")
while True:
    command = input("\nEnter Command: ")
    client.send(command.encode())
    response = client.recv(1024).decode()
    print("Server:", response)
    if command.lower() == "exit":
```

```
        break
client.close()
```

SAMPLE OUTPUT:

```
Connected Successfully

Available Commands
1. hello
2. time
3. date
4. exit

Enter Command: hello
Server: Hello from Server!

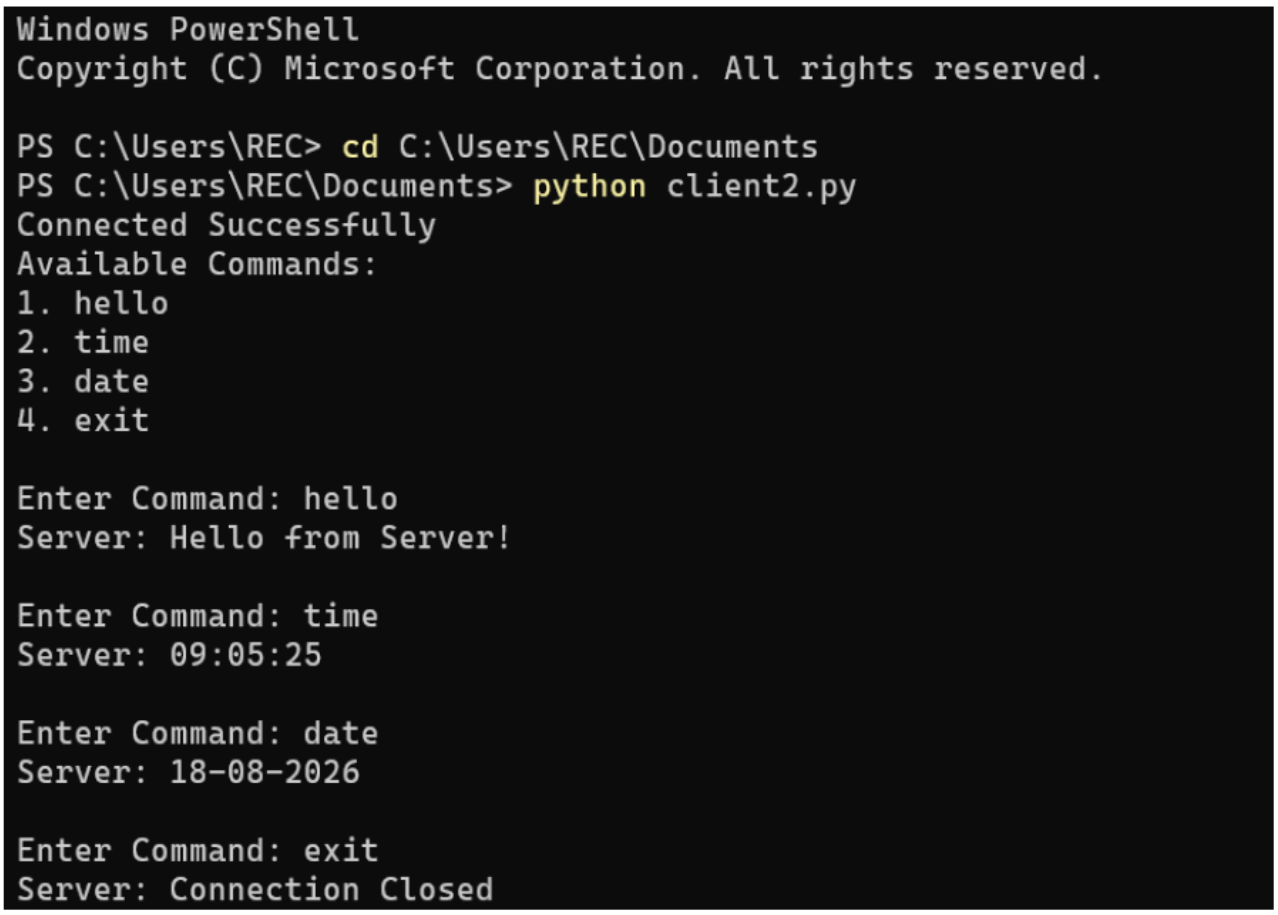
Enter Command: time
Server: 10:35:42

Enter Command: date
Server: 09-07-2026

Enter Command: test
Server: Invalid Command

Enter Command: exit
Server: Connection Closed
```

OUTPUT:



RESULT:

Thus, a TCP socket client–server program was developed and executed successfully to establish communication and exchange messages between the client and server.

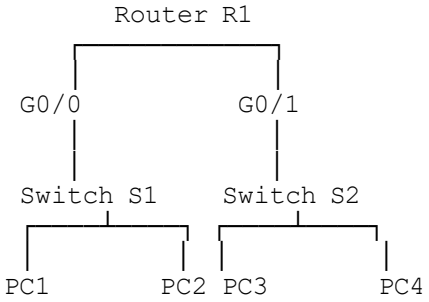
Ex.No.10	Design a simple topology and configure with one router, two switches and PCs using Cisco Packet Tracer
DATE:	

AIM:

To design and configure a simple network topology with one router, two switches, and four PCs in Cisco Packet Tracer for successful inter-network communication.

PROCEDURE:

1. Topology



2. Devices to place

In **Cisco Packet Tracer**, place:

- **1 Router** – Cisco 1941
- **2 Switches** – Cisco 2960
- **4 PCs**

Use **Copper Straight-Through** cables.

3. IP Addressing

Device	Interface	IP Address	Subnet Mask	Gateway
R1	G0/0	192.168.1.1	255.255.255.0	—
R1	G0/1	192.168.2.1	255.255.255.0	—
PC1	NIC	192.168.1.10	255.255.255.0	192.168.1.1
PC2	NIC	192.168.1.11	255.255.255.0	192.168.1.1
PC3	NIC	192.168.2.10	255.255.255.0	192.168.2.1
PC4	NIC	192.168.2.11	255.255.255.0	192.168.2.1

So we have **two different networks**:

```
Network 1: 192.168.1.0/24
       PC1 ─┐
       PC2 ─┤
            S1 ── R1 G0/0 = 192.168.1.1
```

```
Network 2: 192.168.2.0/24
        PC3 ─┐
        PC4 ─┤
             S2 ─── R1 G0/1 = 192.168.2.1
```

4. Configure Router R1

Click **R1 → CLI** and enter:

```
enable
configure terminal

interface gigabitEthernet 0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit

interface gigabitEthernet 0/1
ip address 192.168.2.1 255.255.255.0
no shutdown
exit

end
write memory
```

The important command is:

```
no shutdown
```

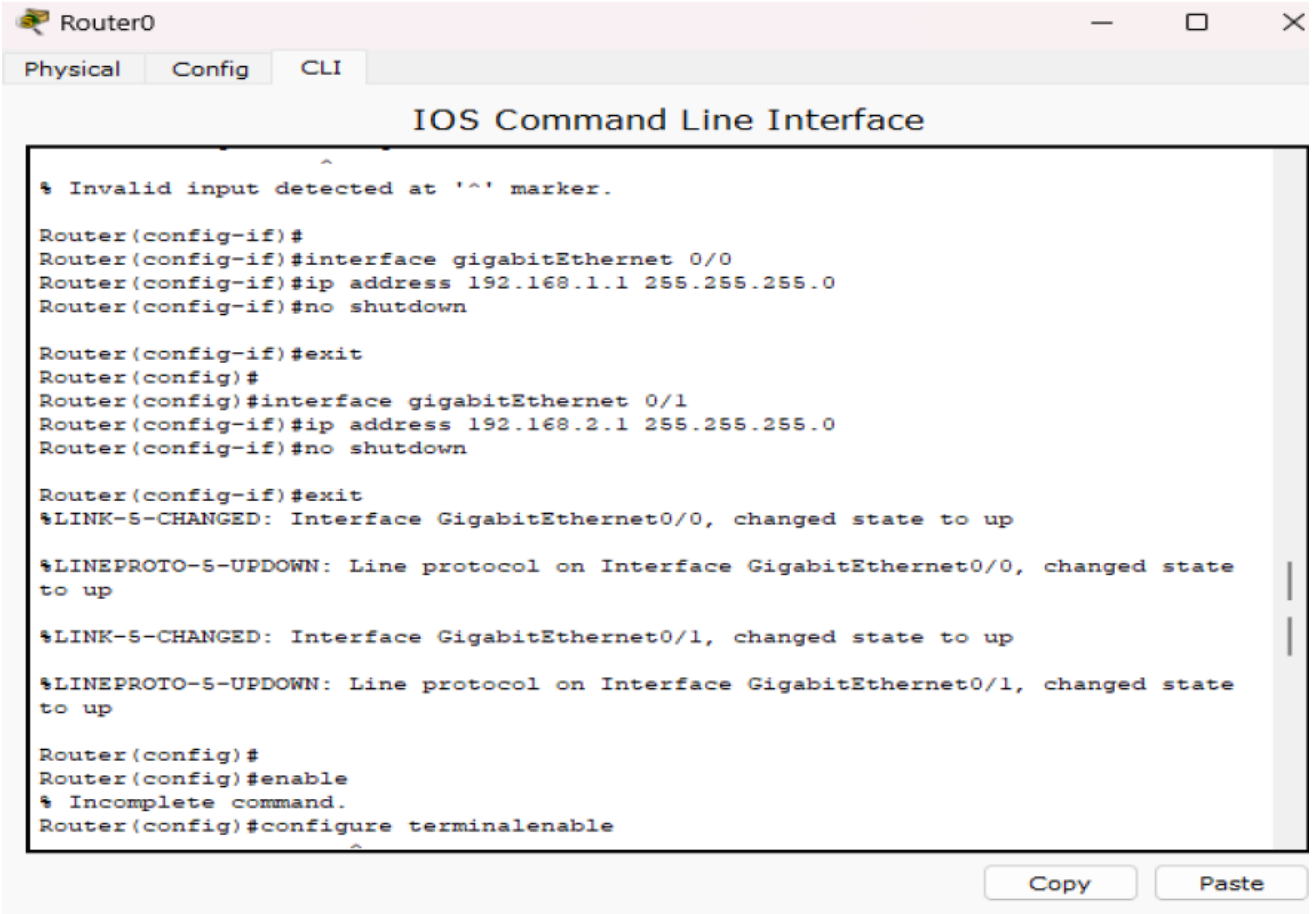
It activates the router interface.

Check the interfaces using:

```
show ip interface brief
```

You should see the interfaces as **up/up**.

SCREENSHOTS:(to be pasted)



5. Configure PC1

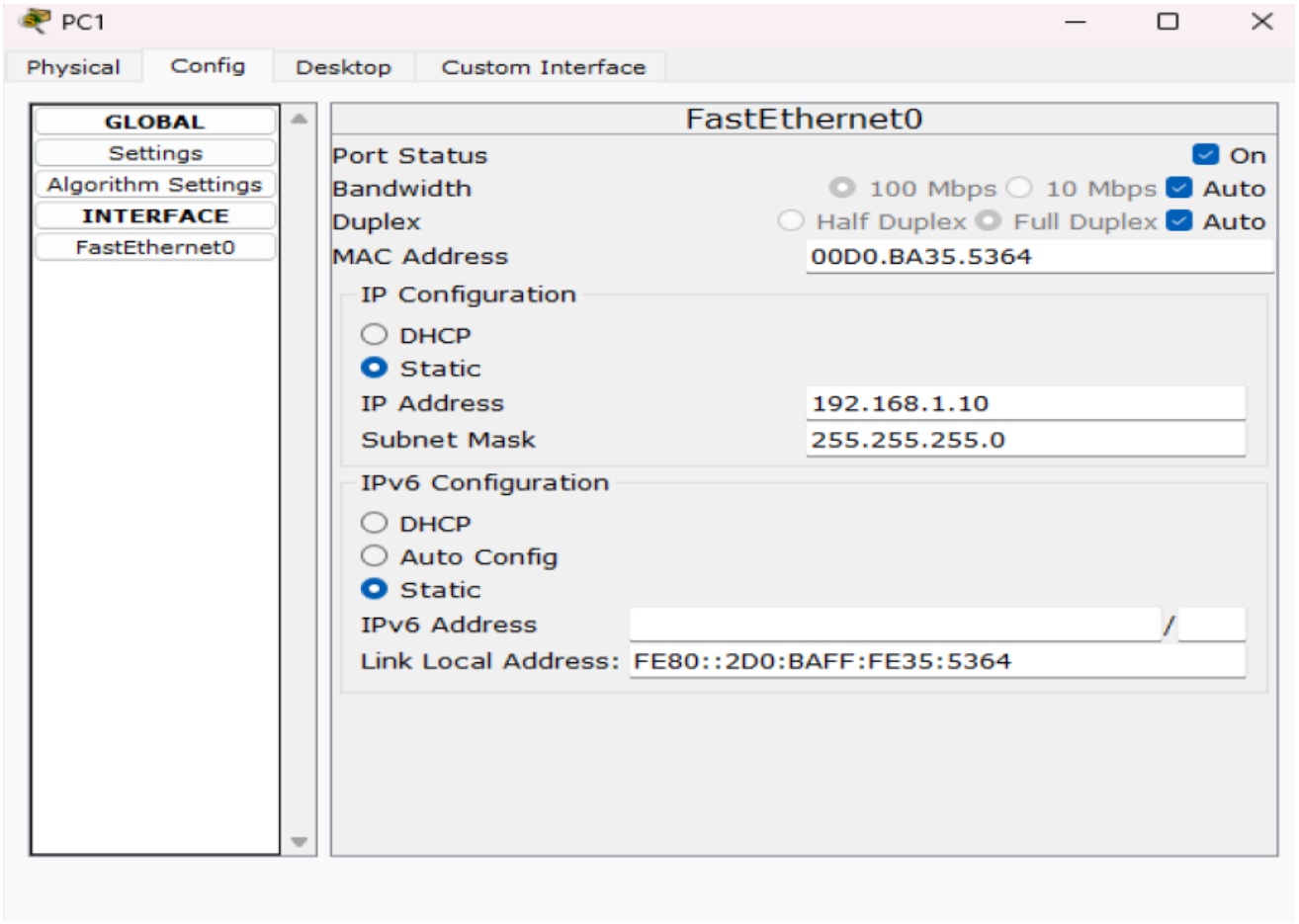
Go to:

PC1 → Desktop → IP Configuration

Enter:

```
IP Address:       192.168.1.10
Subnet Mask:      255.255.255.0
Default Gateway:  192.168.1.1
```

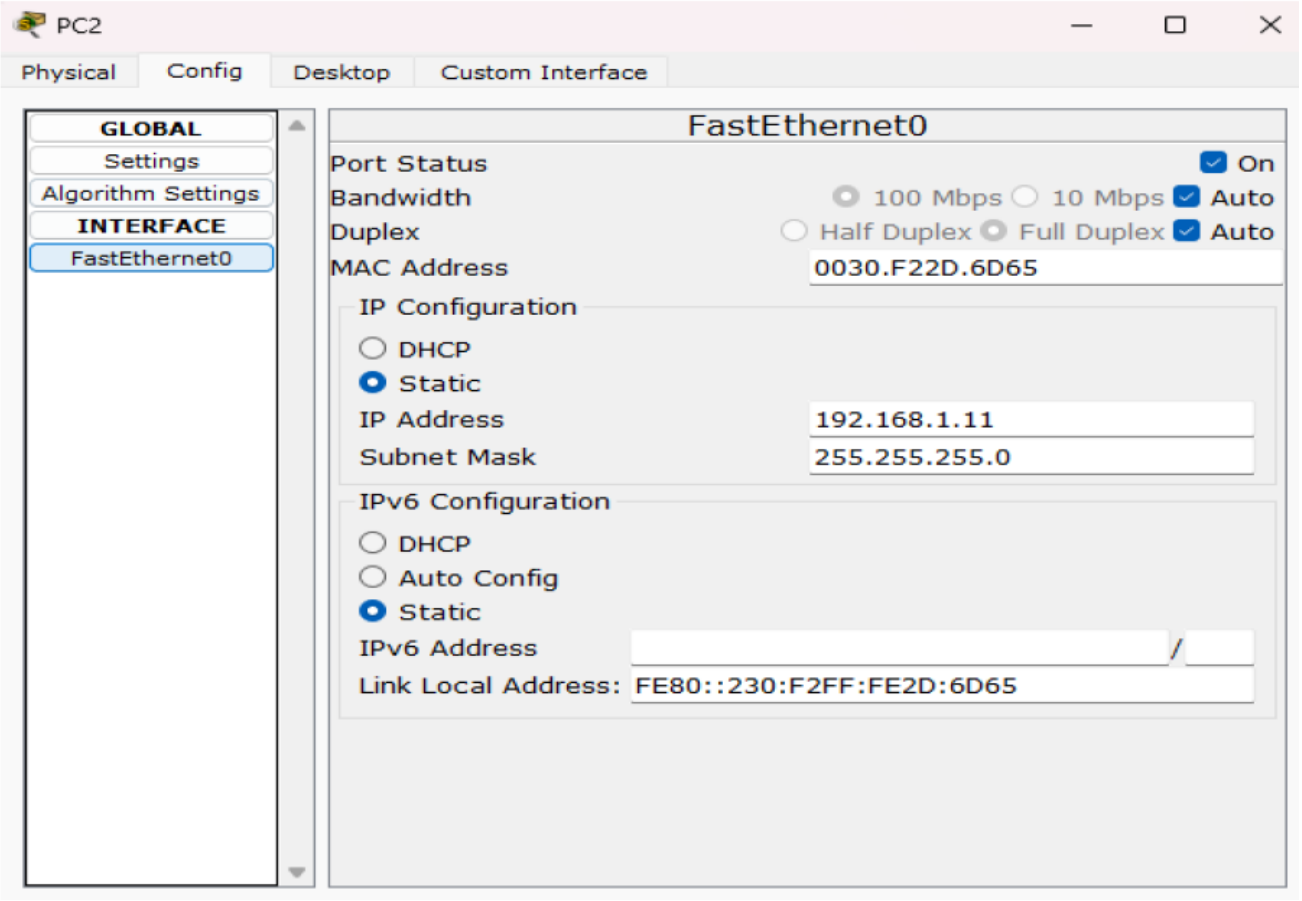
SCREENSHOTS:(to be pasted)



6.Configure PC2

```
IP Address:        192.168.1.11
Subnet Mask:       255.255.255.0
Default Gateway:   192.168.1.1
```

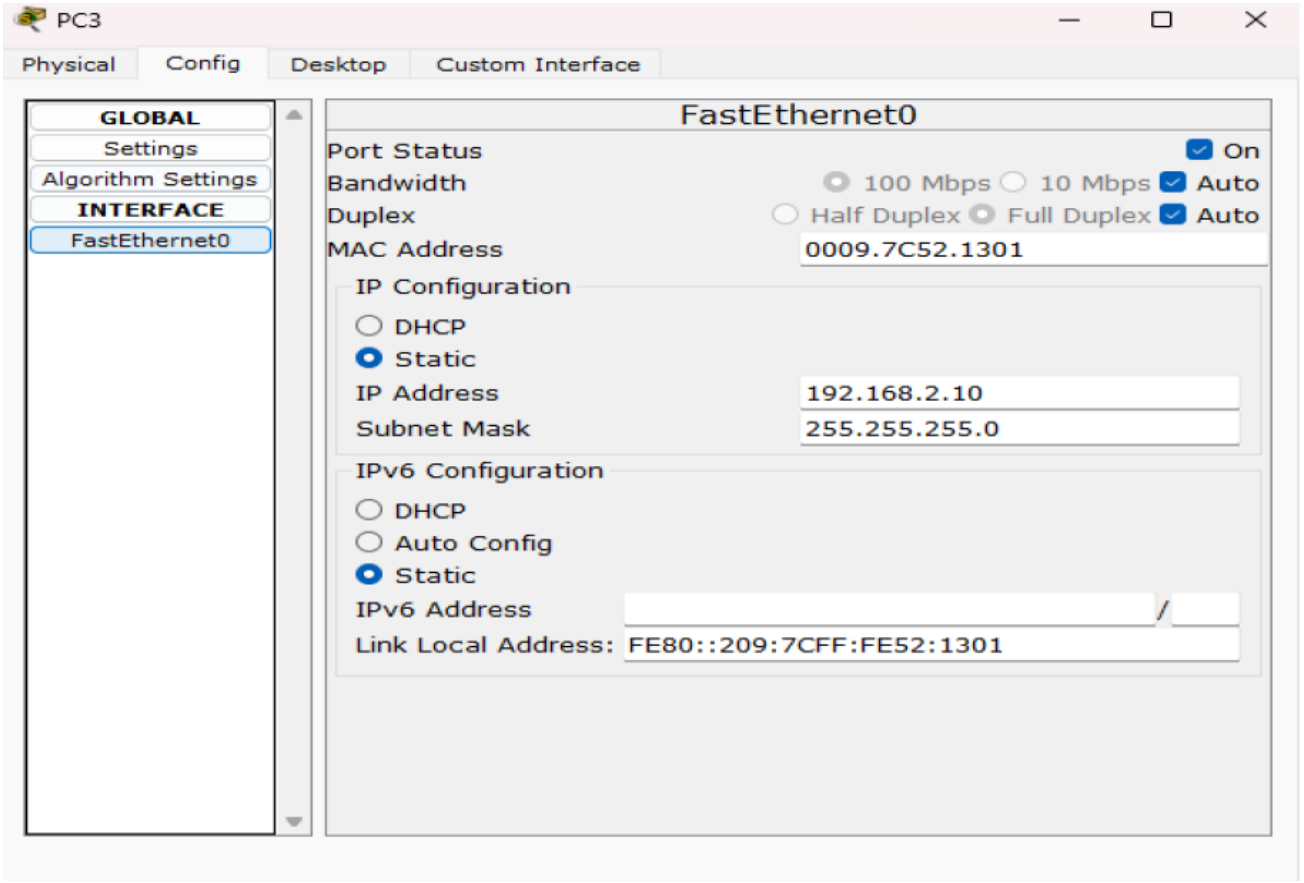
SCREENSHOTS:(to be pasted)



7.Configure PC3

```
IP Address:        192.168.2.10
Subnet Mask:       255.255.255.0
Default Gateway:   192.168.2.1
```

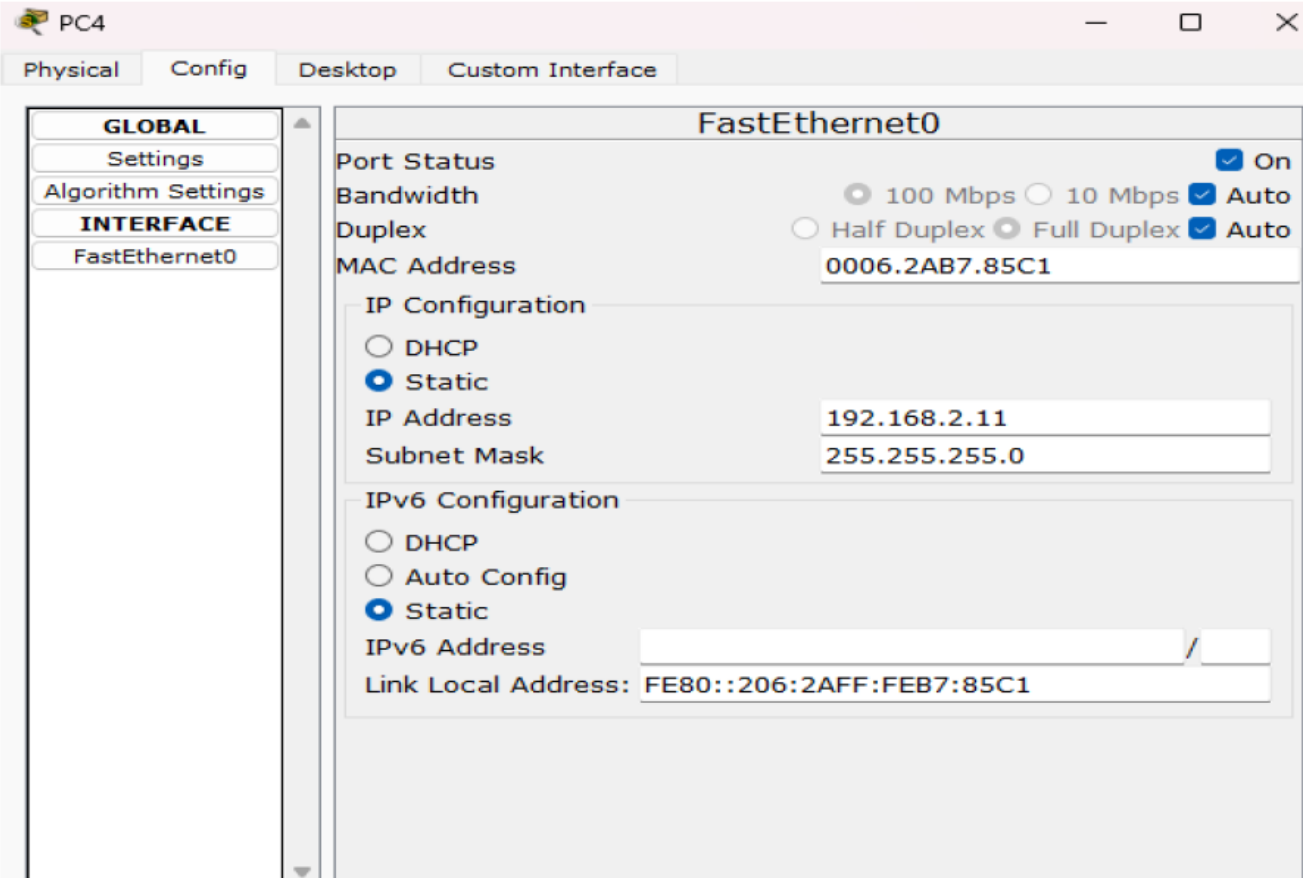
SCREENSHOTS:(to be pasted)



8.Configure PC4

```
IP Address:       192.168.2.11
Subnet Mask:      255.255.255.0
Default Gateway:  192.168.2.1
```

SCREENSHOTS:(to be pasted)



9.Test the network

From **PC1 → Desktop → Command Prompt**, type:

```
ping 192.168.1.11
```

This tests PC1 → PC2.

Then:

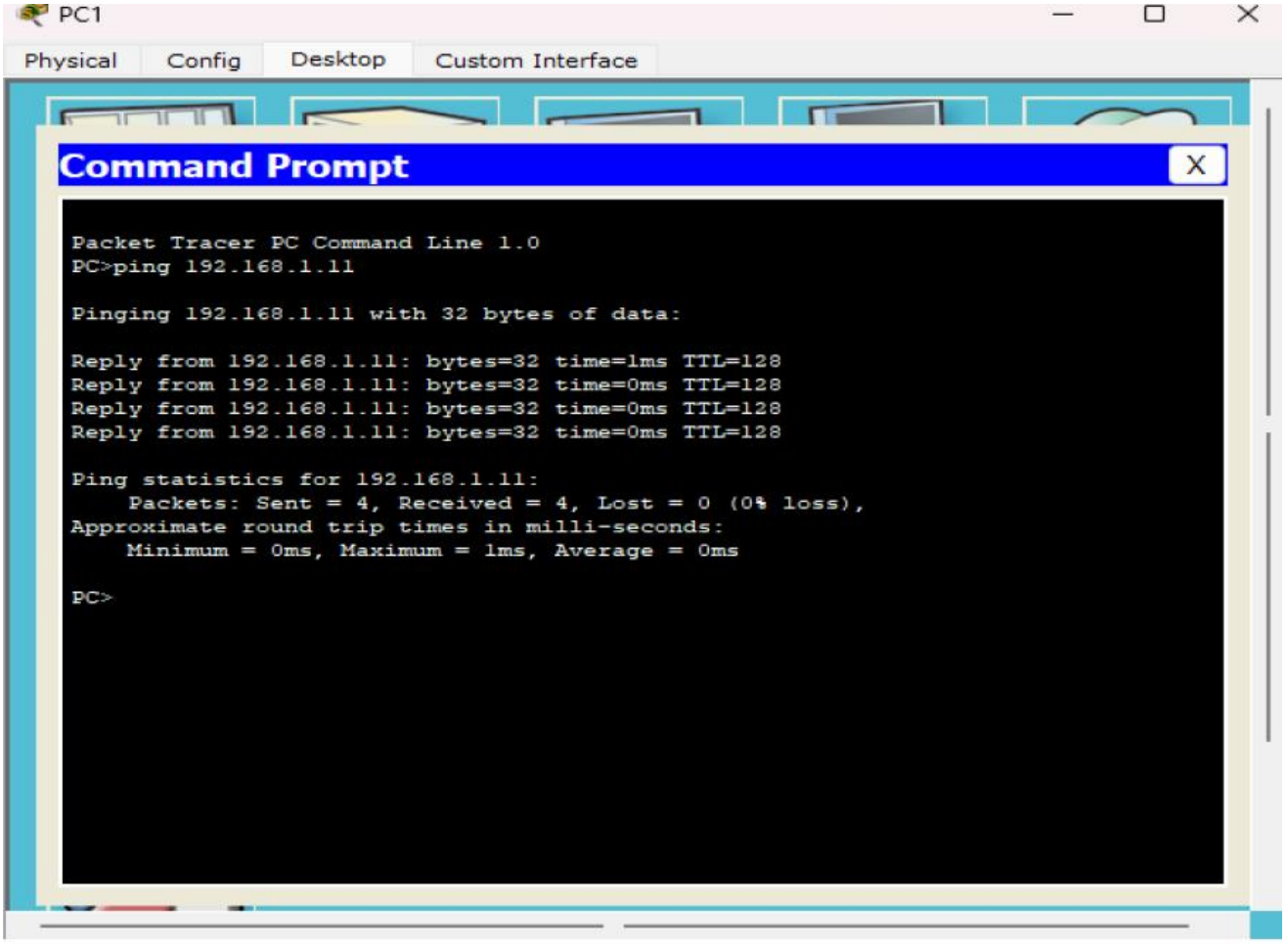
```
ping 192.168.2.10
```

This tests **PC1 → Router → Switch 2 → PC3**.

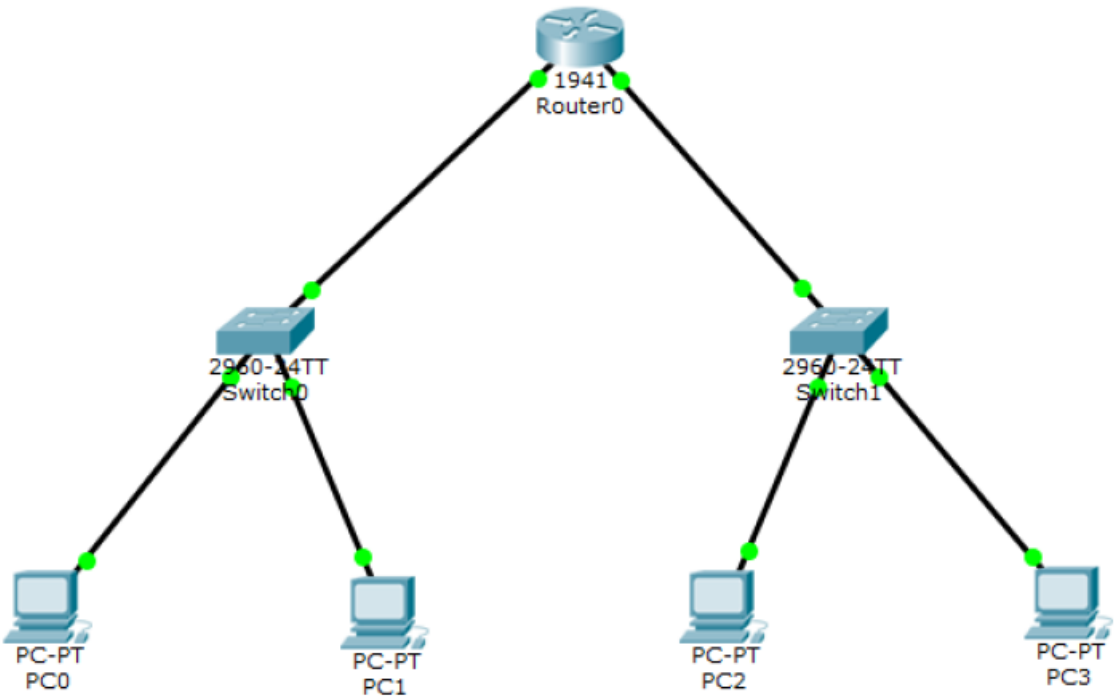
You should receive:

```
Reply from 192.168.2.10
Reply from 192.168.2.10
Reply from 192.168.2.10
Reply from 192.168.2.10
```

SCREENSHOTS:(to be pasted)

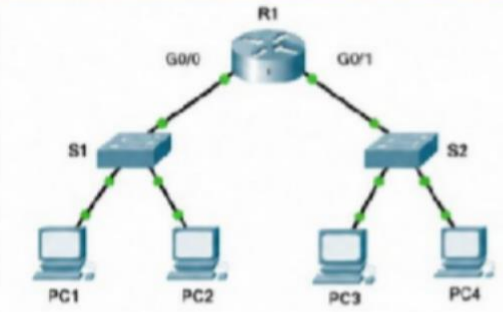


OUTPUT:


1941
Router0
2960-24TT
Switch0
2960-24TT
Switch1
PC-PT
PC0
PC-PT
PC1
PC-PT
PC2
PC-PT
PC3

SAMPLE CONFIGURATION:

TOPOLOGY: One Router, Two Switches and PCs



IP ADDRESSING TABLE

Device	Interface	IP Address	Subnet Mask	Default Gateway
R1	G0/0	192.168.1.1	255.255.255.0	-
R1	G0/1	192.168.2.1	255.255.255.0	-
PC1	NIC	192.168.1.10	255.255.255.0	192.168.1.1
PC2	NIC	192.168.1.11	255.255.255.0	192.168.1.1
PC3	NIC	192.168.2.10	255.255.255.0	192.168.2.1
PC4	NIC	192.168.2.11	255.168.255.0	192.168.2.1

CONFIGURATION STEPS

1. Place Devices

- 1 Router (e.g., 1941)
- 2 Switches (e.g., 2960)
- 4 PCs
- Connections: Copper Straight-Through

2. Configure Router (R1)

```
R1> enable
R1# configure terminal
R1(config)# interface g0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# interface g0/1
R1(config-if)# ip address 192.168.2.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# end
R1# write memory
```

3. Configure PCs

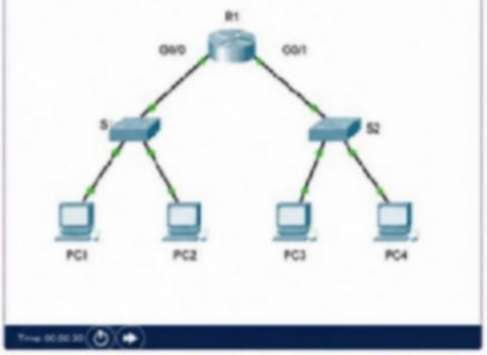
For PC1
- IP Address: 192.168.1.10
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.1.1

For PC2
- IP Address: 192.168.1.11
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.1.1

For PC3
- IP Address: 192.168.2.10
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.2.1

For PC4
- IP Address: 192.168.2.11
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.2.1

PACKET TRACER TOPOLOGY VIEW



VERIFY CONNECTIVITY

From any PC, open Command Prompt and ping other PCs in the same network and across networks.
Example: From PC1 ping PC3 → ping 192.168.2.10
All pings should be successful.

RESULT:

Thus, a network topology consisting of one router, two switches, and four PCs was designed and configured using Cisco Packet Tracer. Successful communication between the networks was verified

Ex.No.11	**STUDY OF SOCKET PROGAMMING IN PYTHON**
Date:	

Aim:
To study the concepts of socket programming in python.

STUDY OF SOCKET PROGAMMING IN PYTHON

A network socket is one endpoint of a two-way communication link between two programs running on a network. It is a software-based handle combining an IP address and a port number, allowing different processes to send and receive data across a device or the internet

Socket programming in python

- Python's core networking library is Socket Module.
- Python's socket module has both class-based and instances-based methods.
- Class-based method is an intuitive approach which doesn't need an instance of a
- socket object.
- For example, in order to print a machine's IP address, you don't need a socket
- object. Instead, just call the socket's class-based methods.
- In instance-based method, if some data needed to be sent to a server application,
- it is more intuitive to create a socket object to perform that explicit operation.
- This module has everything you need to build socket servers and clients.

Algorithm 1: TCP Server

Input: Client message
Output: Response message to the client

1. **Start**
2. Import the socket module.
3. Create a TCP socket using socket().
4. Bind the socket to the IP address localhost and port number 5000.
5. Put the server socket into listening mode using listen().
6. Wait for a client connection using accept().
7. Receive the message from the client using recv().
8. Decode the received message.
9. Display the client's message.
10. Create a response message.

11. Encode and send the response to the client using send().
12. Close the client connection.
13. Close the server socket.
14. **Stop**

TCP Client

Input: Message to be sent to the server
Output: Response received from the server

1. **Start**
2. Import the socket module.
3. Create a TCP socket using socket().
4. Connect to the server using connect() with localhost and port 5000.
5. Create a message to send to the server.
6. Encode the message.
7. Send the message to the server using send().
8. Wait for the server's response using recv().
9. Decode the received response.
10. Display the server's response.
11. Close the client socket.
12. **Stop**

PROGRAM(Server):

```
# TCP Server Program
import socket
# Create a socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# Bind IP address and port
server_socket.bind(("localhost", 5000))
# Listen for client connection
server_socket.listen(1)
print("Server is waiting for connection...")
# Accept client connection
conn, address = server_socket.accept()
print("Client connected:", address)
# Receive message from client
message = conn.recv(1024).decode()
print("Message from client:", message)
# Send response to client
response = "Hello Client! Message received successfully."
conn.send(response.encode())
# Close connection
conn.close()
server_socket.close()
```

PROGRAM (CLIENT):

```
# TCP Client Program

import socket

# Create a socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Connect to server

client_socket.connect(("localhost", 5000))

# Send message to server

message = "Hello Server!"

client_socket.send(message.encode())

# Receive response from server

response = client_socket.recv(1024).decode()

print("Message from server:", response)

# Close connection

client_socket.close()
```

SAMPLE OUTPUT:

Server Output

```
C:\Python> python server.py

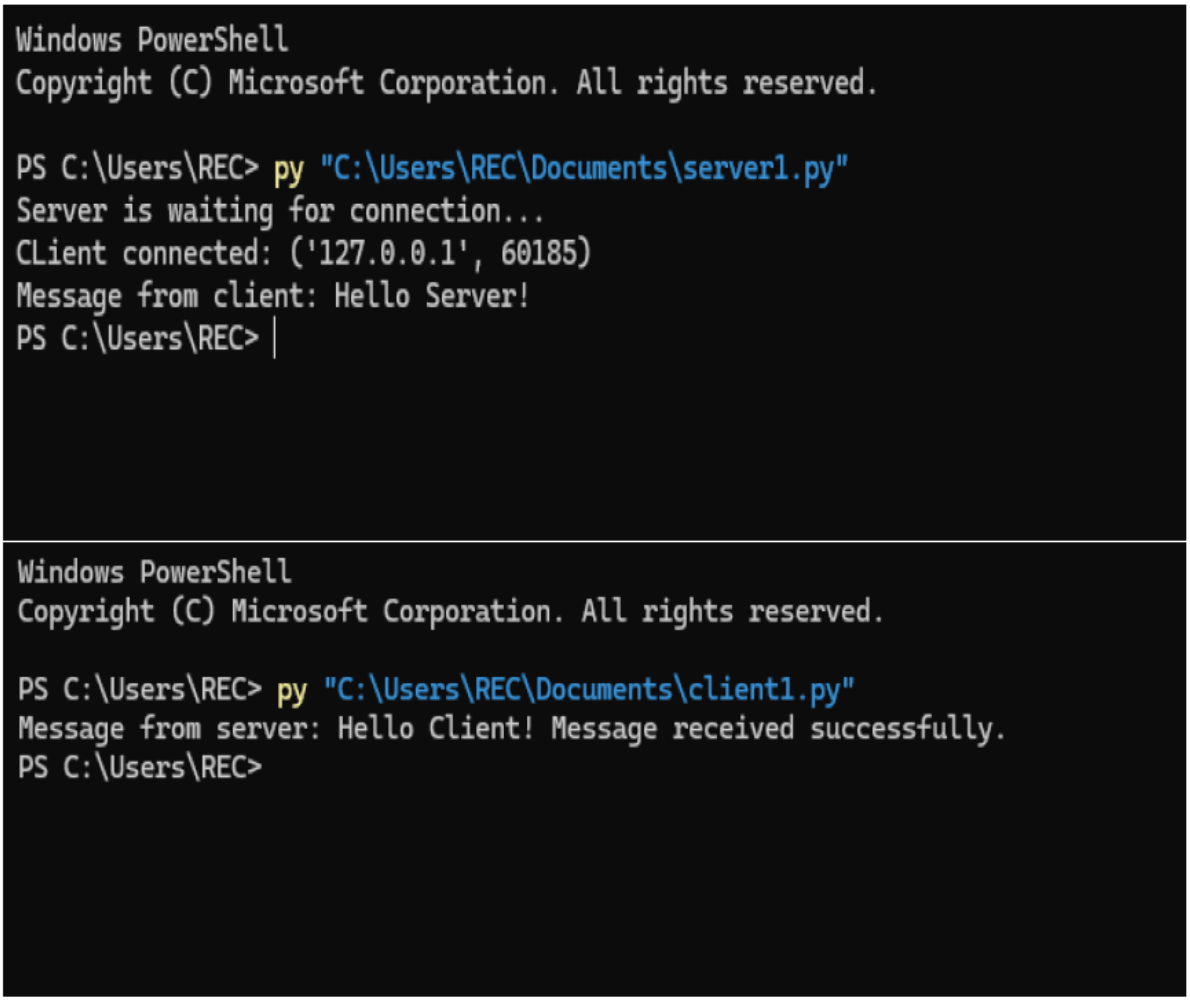
Server is waiting for connection...
Client connected: ('127.0.0.1', 54321)
Message from client: Hello Server!
```

Client Output

```
C:\Python> python client.py

Message from server: Hello Client! Message received successfully.
```

OUTPUT :



RESULT:

Thus, TCP socket programming was studied and client–server communication was successfully established using Python.

Ex. No:12	NETWORK FORENSICS USING PCAPXRAY TOOL
DATE:	

PcapXray is a **network forensics tool**. Its main purpose is to take a captured network traffic file (`.pcap` or `.pcapng`) and help you understand **what happened on the network**

AIM:

To analyze a PCAP file using PcapXray and visualize network activity for basic network forensic investigation.

ALGORITHM: For Installing PCAP and Analyzing:

1. Install PcapXray dependencies using: sudo pip3 install -r requirements.txt
2. Launch the tool with: sudo python3 PcapXray.py
3. Open the web interface and upload the given PCAP file.
4. Click Analyze to process the packet data.
5. View the generated network topology and communication graph.
6. Examine hosts, protocols, and suspicious connections.
7. Save the generated report and diagrams for documentation.

Working entirely in Chrome without installing:

https://wiki.wireshark.org/SampleCaptures

What to do

1. Open the link above.
2. Find **ipv4frags.pcap**.
3. Download it to your **Downloads** folder.
4. Open your online PCAP analyzer.
5. Click **Upload**.
6. Select:

   ipv4frags.pcap

7. Click **Analyze**.

Working entirely in Chrome without installing:

Yes. Let me show you **exactly what to click and what to look for** in A-Packets. You can do this entirely in Chrome.

Step 1 — Open A-Packets

Open the official upload page:

A-Packets PCAP Upload

You should see **“Choose a PCAP”** / **“Click or Drop to Select.”** A-Packets currently accepts `.pcap`, `.pcapng`, and `.cap` files, with a **25 MB free limit**.

Step 2 — Get a sample PCAP

For your first experiment, use a **sample PCAP**, not your personal network traffic.

You can download `ipv4frags.pcap` from the Wireshark sample captures collection:

Wireshark Sample Captures

Save the file in your **Downloads** folder.

Step 3 — Upload the file

On A-Packets:

Click/Drop to Select

Then:

Downloads → ipv4frags.pcap → Open

You should see the filename appear on the page.

Then click:

Upload PCAP and Analyze

A-Packets says the analysis normally takes only a short time, depending on the capture.

Step 4 — Examine the results

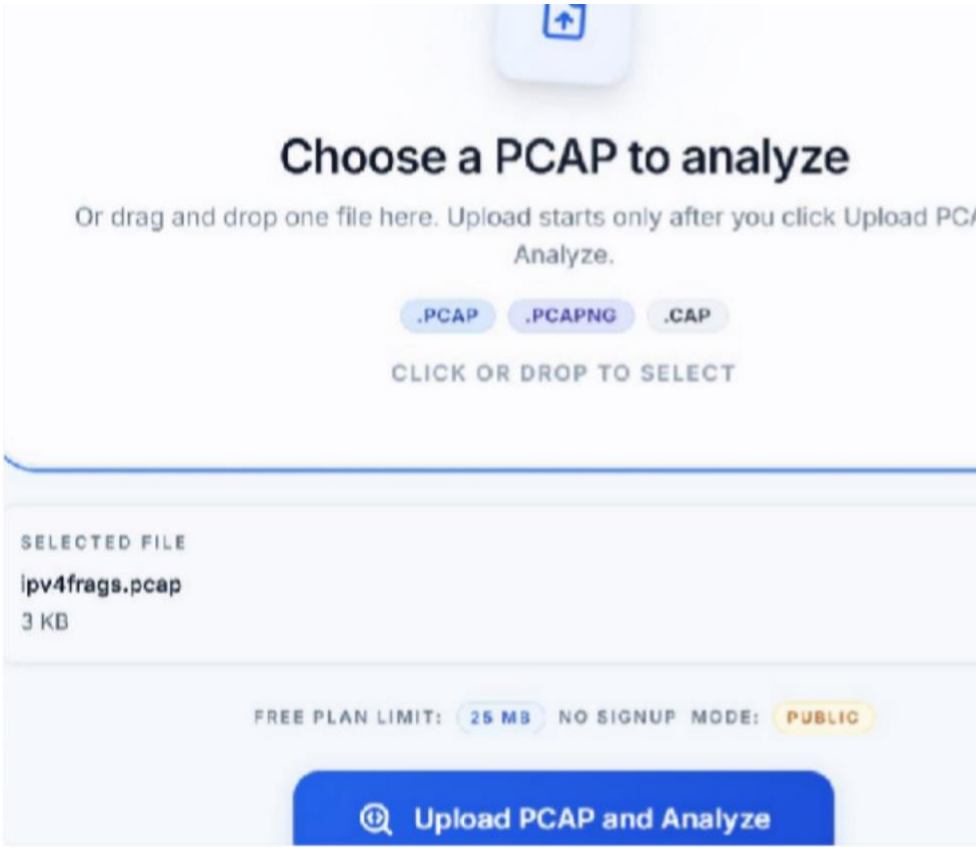
After processing, you'll get a results page.

Summary:

This gives you an overview of the PCAP, such as:

- Number of packets
- Capture duration
- Traffic information
- Hosts involved

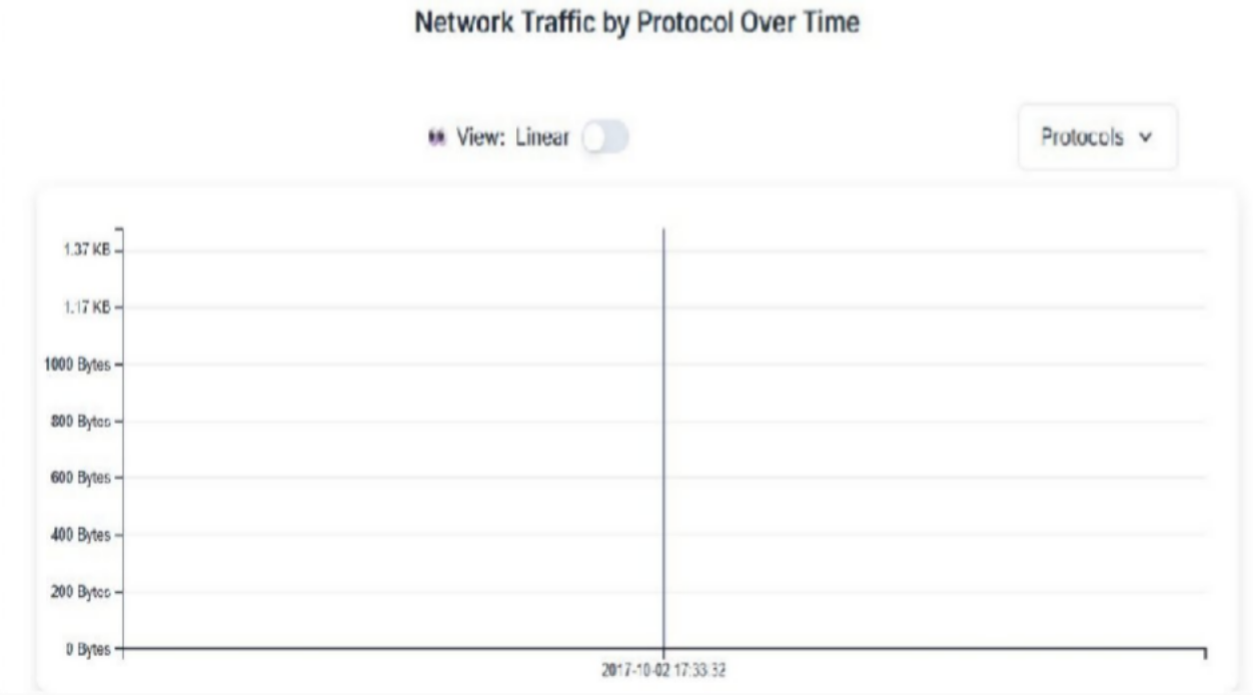
The A-Packets documentation describes a summary and traffic analysis view

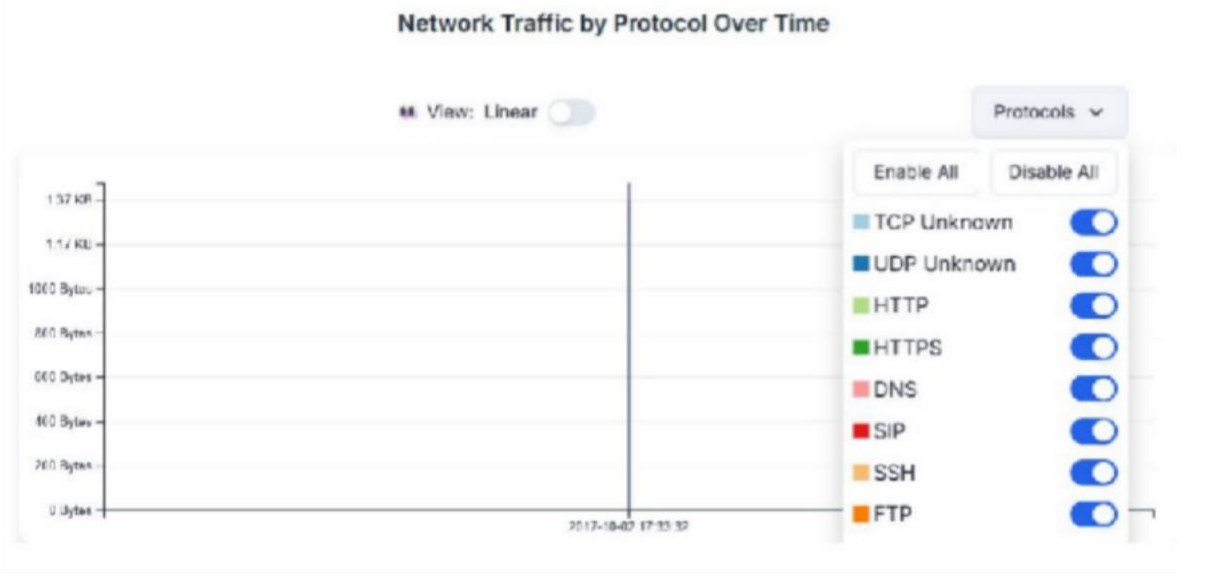

Choose a PCAP to analyze
Or drag and drop one file here. Upload starts only after you click Upload PC
Analyze.
.PCAP
.PCAPNG
.CAP
CLICK OR DROP TO SELECT
SELECTED FILE
ipv4frags.pcap
3 KB
FREE PLAN LIMIT:
25 MB
NO SIGNUP
MODE:
PUBLIC
Upload PCAP and Analyze

After Uploading PCAP View Report:

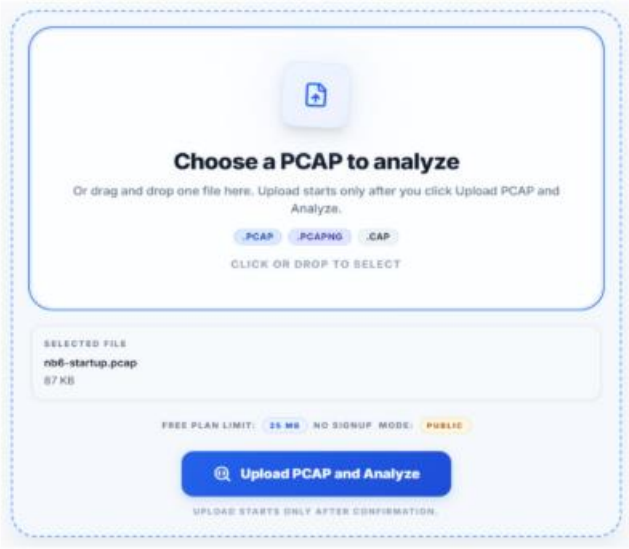
SAMPLE OUTPUTS:

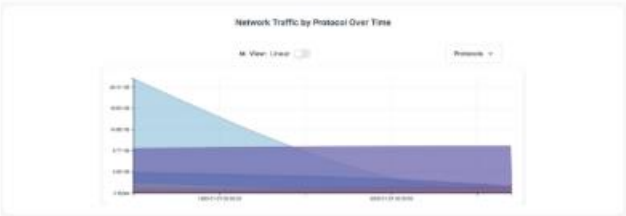


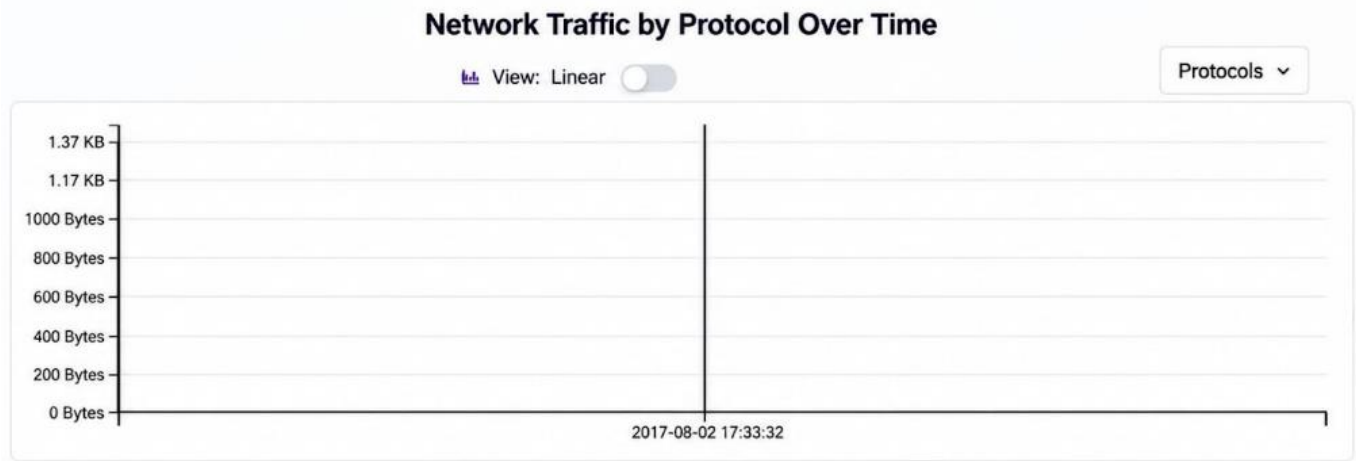


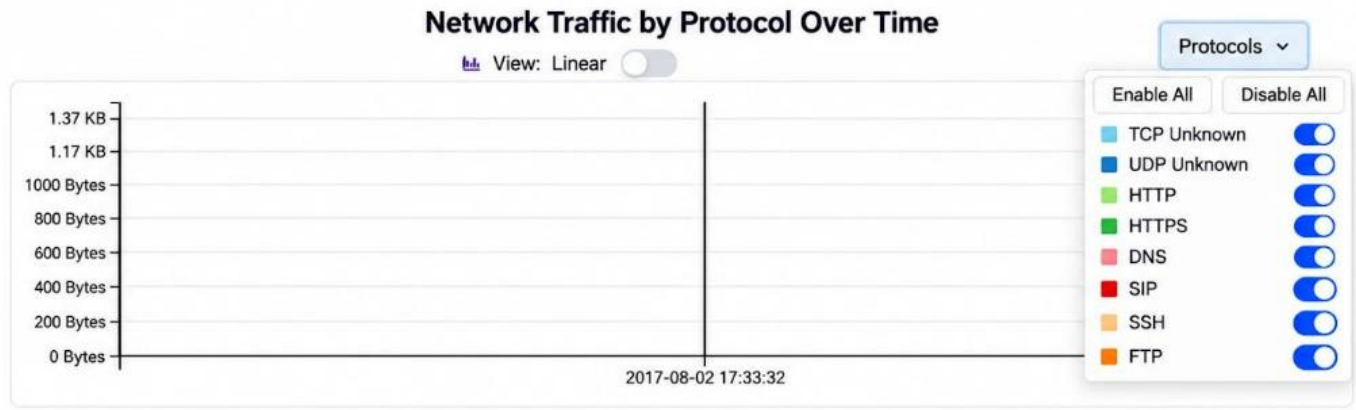


OUTPUT:









RESULT:

The PCAP file was successfully analyzed using PcapXray, and the network activities, communicating hosts, and protocols were identified and visualized.

Ex.No.13	**To Analyze the different types of servers using Webalizer tool**
DATE:	

Aim

To **install, configure, and execute the Webalizer log analysis tool on a Windows operating system** to analyze raw server logs and generate interactive HTML dashboards containing traffic graphs and usage statistics.

Apparatus Required

- **Operating System:** Windows 10 or Windows 11 (64-bit)
- **Software Bundle:** XAMPP for Windows (includes Apache Web Server and pre-configured Webalizer)
- **Access Level:** Windows Administrator privileges
- **Web Browser:** Microsoft Edge, Google Chrome, or Mozilla Firefox

Theory

Server log files record every user transaction, but they are difficult to read in their raw .log text formats. **Webalizer** solves this by converting text logs into structured HTML pages.

While Webalizer is natively a Linux tool, it is seamlessly integrated into Windows environments using **XAMPP**. XAMPP automatically pipes the Apache web server log (access.log) into the Webalizer engine. Webalizer then calculates key performance indicators:

1. **Hits:** Total number of files requested (images, text, pages).
2. **Files:** Total number of successful requests fulfilled by the server.
3. **Pages:** Pure HTML or PHP document views (ignoring individual embedded images).
4. **Visits:** Distinct user journeys tracked via unique IP addresses.

Step-by-Step Procedure

Step 1: Install the XAMPP Server Environment

1. Download the XAMPP installer for Windows from the official website.
2. Double-click the installer executable file.
3. During the **Select Components** prompt, ensure that **Apache** and **Webalizer** are checked.
4. Complete the wizard to install XAMPP into the default directory: C:\xampp\.

Step 2: Start the Web Server

1. Open the Windows Start Menu, search for **XAMPP Control Panel**, and run it as an **Administrator**.
2. Click the **Start** button next to the **Apache** module. The module background will turn green, indicating the web server is actively running and generating log files.

Step 3: Locate the Log and Configuration Files

Verify where Windows is storing your logs and how Webalizer is targeting them:

- **Raw Server Logs Location:** C:\xampp\apache\logs\access.log
- **Webalizer Configuration File Location:** C:\xampp\webalizer\webalizer.conf
  (Optional Tuning): If you open webalizer.conf using Notepad, you will observe that XAMPP has pre-written the paths using forward-slash formatting compatible with the tool:

  text
  LogFile c:/xampp/apache/logs/access.log
  OutputDir c:/xampp/htdocs/webalizer
  Use code with caution.

Step 4: Execute Webalizer Manually via Windows Command Prompt

1. Press Win + R, type cmd, and hit Enter to launch the **Command Prompt**.
2. Change the directory to the XAMPP Webalizer folder:
   cmd
   cd C:\xampp\webalizer
   Use code with caution.
3. Run the executable tool to process the active Windows logs:
   cmd
   webalizer.exe
   Use code with caution.
   The command prompt will flash an information line confirming how many lines of text data it successfully read and converted.

Step 5: View and Analyze the HTML Reports

1. Launch your web browser.
2. Type the following address into the URL bar and press enter:
   http://localhost/webalizer/
3. Click on the current calendar month link to view the visual traffic analysis charts, total system bandwidth usage, and data tables tracking user visits.

Step 6: Automate the Log Analysis using Windows Task Scheduler

To avoid running the command prompt manually every day, set up a background Windows Task:

1. Press the Windows key, search for **Task Scheduler**, and open it.
2. In the right-hand Actions menu, select **Create Basic Task**.
3. Name your task XAMPP Webalizer Update and select **Daily** as the frequency trigger.
4. Set the Action to **Start a program**.
5. Click **Browse** and target the executable path: C:\xampp\webalizer\webalizer.exe
6. In the **Start in (optional)** textbox, type C:\xampp\webalizer to ensure the program finds its default configuration parameters. Click **Finish**.

Conclusion

The Webalizer analysis program was successfully deployed and executed on the Windows OS environment via XAMPP. Raw, unorganized data entries in access.log were converted into comprehensive, easy-to-read web graphs. The setup provides administrators with a highly

visual framework to evaluate server stress, incoming traffic spikes, and user distribution patterns.

Ex.No.14	Develop a customized ping command to test the server connectivity
DATE:	

14.1 Customized ping command to test the server connectivity
Aim:

To develop a customized ping command to test the server connectivity.

Algorithm for Program1: Customized ping command to test the server connectivity:

1. Start.
2. Import the `socket` and `time` modules.
3. Specify the server name, port number, and number of attempts.
4. Create a TCP socket.
5. Set the timeout period.
6. Record the start time.
7. Connect to the server.
8. Record the end time.
9. Calculate and display the response time.
10. If the connection fails, display "Request timed out".
11. Repeat Steps 4–10 for the specified number of attempts.
12. Stop.

PROGRAM 1:

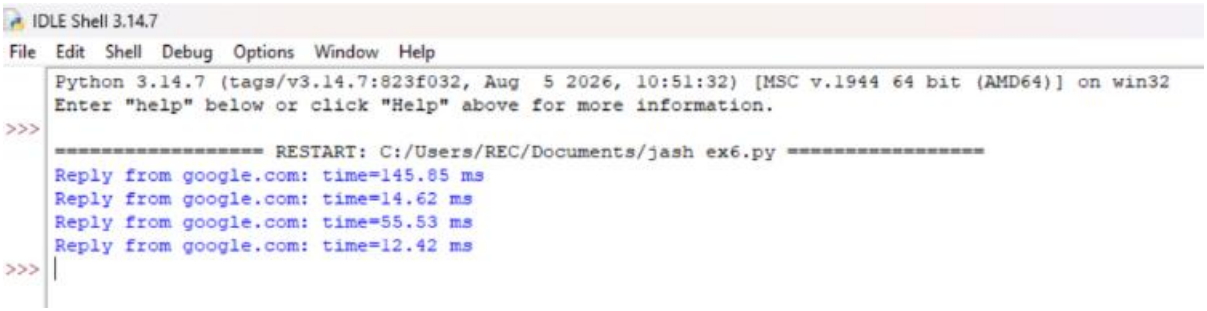
Customized ping command to test the server connectivity:

```
import socket
import time
host = "google.com"
port = 80
count = 4
for i in range(count):
    try:
        # Create a socket
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        # Set timeout (optional)
        s.settimeout(3)
        # Record start time
        start = time.time()
        # Connect to the server
        s.connect((host,  port))
        # Record end time
        end = time.time()
        # Close the socket
        s.close()
        print(f"Reply from {host}: time={(end - start) * 1000:.2f} ms")
    except Exception:
        print("Request timed out")
```

Sample Output:

Reply from google.com: time=18.52 ms
Reply from google.com: time=17.84 ms
Reply from google.com: time=19.03 ms
Reply from google.com: time=18.26 ms

OUTPUT



RESULT:

Thus, a customized ping command was developed and executed successfully to test server connectivity and measure response time.

14.2 Customized Ping Program to Measure Min, Max, and Average RTT

AIM:

To develop a Customized Ping Program to Measure Min, Max, and Average RTT

ALGORITHM for Program 2:

- Start.
- Import the required modules.
- Initialize the server address, port number, and number of ping attempts.
- Create an empty list to store RTT values.
- Repeat the following for each ping attempt:

    - Create a TCP socket.
    - Set the timeout value.
    - Record the start time.
    - Connect to the server.
    - Record the end time.
    - Calculate the RTT and store it in the list.
    - Display the RTT.
    - If the connection fails, display **"Request timed out"**.

- Calculate and display:

    - Packets Sent
    - Packets Received
    - Packets Lost
    - Minimum RTT
    - Maximum RTT
    - Average RTT

- Stop.

PROGRAM2: Customized Ping Program to Measure Min, Max, and Average RTT

```
import socket
import time
# Server details
host = "google.com"
port = 80
count = 4
# List to store RTT values
rtt_list = []
print(f"Pinging {host} with TCP packets:\n")
for i in range(count):
    try:
        # Create a TCP socket
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(3)
        # Record start time
        start = time.time()
        # Connect to the server
        s.connect((host,  port))
        # Record end time
        end = time.time()
        # Close the socket
        s.close()
        # Calculate RTT in milliseconds
        rtt = (end - start) * 1000
        rtt_list.append(rtt)
        print(f"Reply {i+1}: time = {rtt:.2f} ms")
    except Exception:
        print(f"Reply {i+1}: Request timed out")
# Display statistics
if len(rtt_list) > 0:
```

```
    print("\n--- Ping Statistics ---")
    print(f"Packets Sent     : {count}")
    print(f"Packets Received : {len(rtt_list)}")
    print(f"Packets Lost     : {count - len(rtt_list)}")
    print(f"Minimum RTT      : {min(rtt_list):.2f} ms")
    print(f"Maximum RTT      : {max(rtt_list):.2f} ms")
    print(f"Average RTT      : {sum(rtt_list)/len(rtt_list):.2f} ms")
else:
    print("\nNo response received from the server.")
```

SAMPLE OUTPUT:

Pinging google.com with TCP packets:

Reply 1: time = 18.24 ms

Reply 2: time = 17.81 ms

Reply 3: time = 19.15 ms

Reply 4: time = 18.56 ms

--- Ping Statistics ---

Packets Sent : 4

Packets Received : 4

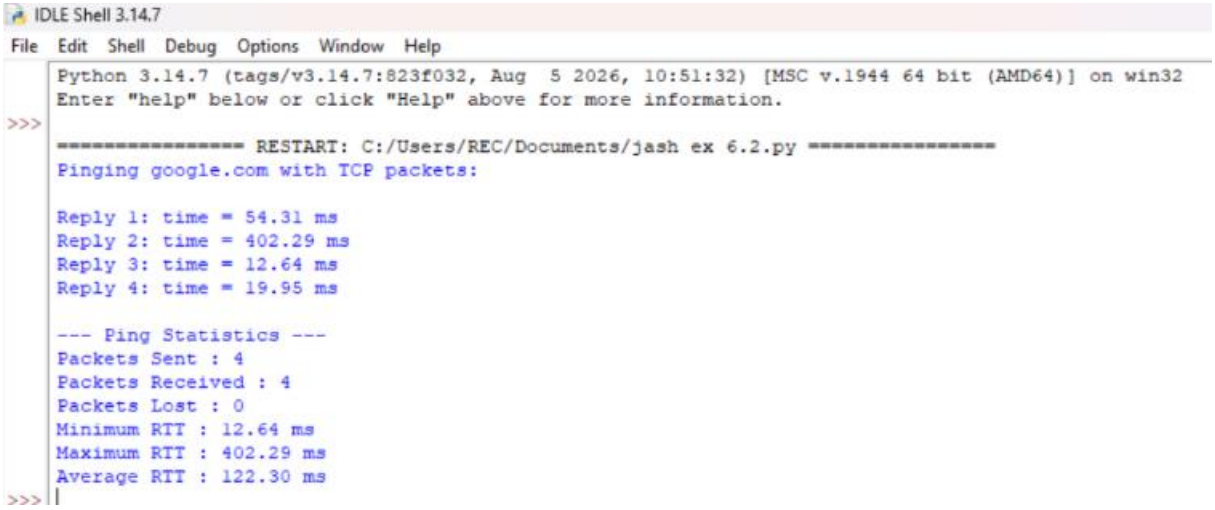
Packets Lost : 0

Minimum RTT : 17.81 ms

Maximum RTT : 19.15 ms

Average RTT : 18.44 ms

OUTPUT:



RESULT:

Thus, a customized ping program was developed and executed successfully to measure and display the minimum, maximum, and average RTT, along with packet statistics.

Ex.No.15	Write a code using RAW sockets to implement packet sniffing.
DATE:	

AIM:

To develop a Python program using raw sockets that captures Ethernet frames and extracts the source and destination MAC addresses and protocol details for network analysis and troubleshooting.

Definition:

Packet sniffing is the technique of capturing and examining network packets transmitted over a network to monitor communication, troubleshoot network issues, and analyze network traffic.

Algorithm:

1. Start.
2. Import the required modules.
3. Get the local host IP address.
4. Create and bind a raw socket.
5. Enable IP header inclusion and promiscuous mode.
6. Set the packet count to **5**.
7. Receive one packet at a time.
8. Extract and display:
   Destination MAC address
   Source MAC address
   Protocol type
   Packet length
9. Repeat until **5 packets** are captured.
10. Disable promiscuous mode.
11. Display the completion message.
12. Stop.

PROGRAM:

```
import socket
import struct

def get_mac_addr(bytes_addr):
    """Convert MAC address from bytes to readable format."""
    return ':'.join('{:02X}'.format(b) for b in bytes_addr)

def get_protocol(bytes_proto):
    """Convert protocol bytes to hexadecimal."""
    return hex(struct.unpack('!H', bytes_proto)[0])

def ethernet_frame(data):
    """Extract Ethernet frame information."""
    dest_mac, src_mac, proto = struct.unpack('!6s6s2s', data[:14])
    return (
        get_mac_addr(dest_mac),
        get_mac_addr(src_mac),
        get_protocol(proto),
        data[14:]
    )
def main():
    host = socket.gethostbyname(socket.gethostname())
    print("Listening on:", host)
    try:
        # Create raw socket
        conn = socket.socket(socket.AF_INET,
                             socket.SOCK_RAW,
                             socket.IPPROTO_IP)

        # Bind socket
        conn.bind((host, 0))

        # Include IP header
        conn.setsockopt(socket.IPPROTO_IP,
                        socket.IP_HDRINCL,
                        1)

        # Enable promiscuous mode
        conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

        print("\nPacket Sniffer Started...\n")

        # Capture only 5 packets
        packet_count = 5

        for i in range(packet_count):

            raw_data, addr = conn.recvfrom(65536)

            print(f"\nPacket {i+1}")
            print("-" * 40)
```

```
            if len(raw_data) >= 14:
                dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)

                print("Destination MAC :", dest_mac)
                print("Source MAC      :", src_mac)
                print("Protocol        :", eth_proto)
                print("Packet Length   :", len(raw_data), "bytes")

        # Disable promiscuous mode
        conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)

        print("\nPacket Sniffer Stopped Successfully.")

    except PermissionError:
        print("Run this program as Administrator.")

    except KeyboardInterrupt:
        conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
        print("\nPacket Sniffer Stopped by User.")

if __name__ == "__main__":
    main()
```

SAMPLE OUTPUT:

Listening on: 192.168.1.10

Packet Sniffer Started...

Packet 1

Destination MAC : 45:00:00:3C:1C:46
Source MAC : 40:06:B1:E6:C0:A8
Protocol : 0x101
Packet Length : 60 bytes

Packet 2

Destination MAC : 45:00:00:34:2F:9A
Source MAC : 40:11:C3:D8:C0:A8
Protocol : 0x101
Packet Length : 52 bytes

Packet 3

Destination MAC : 45:00:00:54:3B:8F
Source MAC : 40:06:91:AB:C0:A8
Protocol : 0x101
Packet Length : 84 bytes

Packet 4

Destination MAC : 45:00:00:28:5D:71
Source MAC : 40:01:6F:12:C0:A8
Protocol : 0x101
Packet Length : 40 bytes

Packet 5

Destination MAC : 45:00:00:3C:7A:15
Source MAC : 40:06:5A:9F:C0:A8
Protocol : 0x101
Packet Length : 60 bytes

Packet Sniffer Stopped Successfully.

OUTPUT:

```
Listening on: 192.168.56.1

Packet Sniffer Started...


Packet 1
----------------------------------------
Destination MAC : 45:00:00:A5:68:82
Source MAC      : 00:00:04:11:65:22
Protocol        : 0xc0a8
Packet Length   : 165 bytes

Packet 2
----------------------------------------
Destination MAC : 45:00:00:A5:68:82
Source MAC      : 00:00:04:11:65:22
Protocol        : 0xc0a8
Packet Length   : 165 bytes

Packet 3
----------------------------------------
Destination MAC : 45:00:00:A5:68:83
Source MAC      : 00:00:04:11:65:21
Protocol        : 0xc0a8
Packet Length   : 165 bytes

Packet 4
----------------------------------------
Destination MAC : 45:00:00:A5:68:83
Source MAC      : 00:00:04:11:65:21
Protocol        : 0xc0a8
Packet Length   : 165 bytes

Packet 5
----------------------------------------
Destination MAC : 45:00:00:A5:68:84
Source MAC      : 00:00:04:11:65:20
Protocol        : 0xc0a8
Packet Length   : 165 bytes

Packet Sniffer Stopped Successfully.
>>>
```

RESULT:

Thus, a packet sniffing program was developed using raw sockets to capture Ethernet frames and extract source MAC address, destination MAC address, protocol type, and packet length.